UNIVERSIDAD TECNOLÓGICA NACIONAL

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

FOOD STORE

BASES DE DATOS II

TRABAJO PRÁCTICO INTEGRADOR

INFORME TÉCNICO FINAL

Modelado NoSQL · Persistencia en Atlas · CRUD y baja lógica · Respaldo y recuperación

Integrantes:

Ricardo Casto Gil — Comisión 1

Macarena Larrosa — Comisión 1

Repositorio del proyecto: github.com/CastoGil/foodstore-tpi-bd2

Índice de contenidos

Contenido Página

Resumen ejecutivo 3

1. Introducción 3

2. Descripción del negocio y alcance 3

3. Objetivos del proyecto 4

4. Arquitectura y tecnologías 4

5. Infraestructura y configuración de MongoDB Atlas 5

6. Modelo de datos documental 8

7. Operaciones CRUD y reglas de negocio 10

8. Mecanismo de respaldo 13

9. Restauración y comprobación de recuperación 14

10. Objetivos de recuperación RTO y RPO 15

11. Resultados de las pruebas y evidencias 15

12. Limitaciones y propuestas de mejora 16

13. Conclusión 16

14. Material complementario 16

ANEXO I. Evidencias complementarias 17

ANEXO II. Fragmentos representativos del código fuente 31

Resumen ejecutivo

Food Store es una aplicación de consola desarrollada con Node.js que utiliza MongoDB Atlas para administrar clientes, productos y pedidos. El proyecto implementa relaciones mediante referencias y documentos embebidos, operaciones CRUD con baja lógica, validación monetaria de importes, un índice único para correos y un mecanismo de resguardo con MongoDB Database Tools.

Las pruebas manuales respaldaron el uso real de Atlas, los módulos CRUD, las validaciones, la preservación del historial y la recuperación en una base independiente. Se documentaron tres respaldos exitosos; se restauraron 18 documentos sin fallos y se verificó el índice email_1. Tras una corrección controlada de un dato de ejemplo, cinco pedidos fueron consistentes en sus subtotales y totales.

Se propusieron RPO de 24 horas y RTO de 60 minutos. Son metas de planificación: el backup se ejecuta manualmente y el proceso integral de recuperación no se cronometró. El documento diferencia estos objetivos de las duraciones medidas e identifica mejoras de seguridad, automatización y pruebas.

03	18	03	05 / 05
COLECCIONES	DOCUMENTOS RESTAURADOS	BACKUPS EXITOSOS	PEDIDOS CONSISTENTES

1. Introducción

Food Store es una aplicación académica de consola desarrollada con Node.js y MongoDB Atlas para gestionar clientes, productos y pedidos de un comercio gastronómico. El trabajo se centra en el diseño de un modelo documental NoSQL, la persistencia en la nube, las operaciones CRUD, la baja lógica y la recuperación de información mediante respaldos.

Se eligió una interfaz de línea de comandos (CLI) para concentrar el desarrollo en el modelo de datos, las reglas de negocio y la comunicación de la aplicación con MongoDB. Esta solución permite comprobar que la información permanece almacenada en Atlas después de finalizar el programa y volver a ejecutarlo.

2. Descripción del negocio y alcance

✓ Problema de negocio

Un comercio de comidas y bebidas necesita administrar un catálogo de productos, registrar los clientes y almacenar los pedidos con sus importes. También necesita consultar operaciones anteriores aun cuando cambien los precios o se desactiven productos del catálogo.

Food Store organiza esta información en tres colecciones relacionadas:

- **Clientes:** datos de identificación y contacto; incluye un control de unicidad del correo electrónico.
- **Productos:** catálogo gastronómico clasificado en `comidas`, `bebidas` y `postres`, con importes en pesos argentinos.
- **Pedidos:** transacciones vinculadas a clientes y productos, con un detalle embebido que conserva los precios utilizados al registrar cada pedido.

✓ Funcionalidades implementadas

La aplicación permite crear, consultar, modificar y desactivar clientes y productos. En pedidos permite crear, listar activos, buscar por identificador, cambiar el estado comercial y aplicar una baja lógica. Los pedidos conservan sus ítems, cantidades, subtotales y total.

Las transiciones comerciales admitidas son `pendiente` → `entregado` y `pendiente` → `cancelado`. El sistema no implementa el estado `en preparación`.

✓ Funcionalidades fuera del alcance

Food Store no incluye portal web, procesamiento de pagos, facturación fiscal, gestión de envíos ni descuento automático de stock. Tampoco se presenta como un sistema de producción de alta disponibilidad o con garantías transaccionales entre colecciones.

3. Objetivos del proyecto

Objetivo general: diseñar, implementar y comprobar una solución de persistencia documental NoSQL en MongoDB Atlas para la gestión básica de Food Store.

Objetivos específicos:

1. Diseñar al menos tres colecciones relacionadas y justificar el uso de **linking** y **embedding**.
2. Aplicar tipos BSON adecuados: `ObjectId`, `Date`, `Decimal128`, `Int32` y `Boolean`.
3. Implementar las operaciones CRUD respetando la baja lógica.
4. Conectar una aplicación Node.js con MongoDB Atlas y demostrar persistencia de los documentos.
5. Validar reglas de negocio, integridad de importes e índice único de correo electrónico.
6. Generar un respaldo con `mongodump`, comprobar su restauración con `mongorestore` y documentar objetivos RTO/RPO sin confundirlos con tiempos medidos.

4. Arquitectura y tecnologías

Componente	Tecnología y función
Interfaz	Consola interactiva en Node.js
Persistencia	MongoDB Atlas, clúster M0 de uso académico
Acceso a datos	Driver nativo mongodb
Configuración	dotenv; variables MONGODB_URI y MONGODB_DB
Cálculos monetarios	decimal.js y almacenamiento Decimal128
Respaldo	scripts/backup.ps1 y MongoDB Database Tools (mongodump)
Recuperación	mongorestore y comprobación con mongosh
Versionado	Git y GitHub

El proyecto se divide en componentes con responsabilidades separadas:

Archivo	Responsabilidad
src/app.js	Menú principal, manejo de errores y cierre
src/config/conexion.js	Apertura y reutilización de MongoClient; validación con ping
src/consola/entrada.js	Lectura de datos de consola
src/operaciones/clientes.js	CRUD y control del correo único
src/operaciones/productos.js	CRUD, categorías y precios
src/operaciones/pedidos.js	Pedidos, referencias, historial y estados
scripts/backup.ps1	Respaldo completo de la base mediante mongodump

✓ Comunicación cliente-servidor

El operador utiliza la CLI de Food Store. Node.js valida los datos y, a través del driver oficial, se conecta de forma segura a MongoDB Atlas para ejecutar las operaciones. Atlas procesa la solicitud y devuelve los documentos o el resultado de la escritura. La aplicación muestra el resultado en consola.

Las funciones asíncronas emplean `async` y `await` para esperar la respuesta de las consultas o modificaciones. La conexión se establece cuando se necesita y se cierra al finalizar el programa.

En términos de operaciones MongoDB: **CREATE** se implementa con `insertOne`, **READ** con `find` y `findOne`, y **UPDATE** y **DELETE lógico** mediante `updateOne` con `$set`. No se realiza eliminación física con `deleteOne` ni `deleteMany` en los módulos CRUD.

5. Infraestructura y configuración de MongoDB Atlas

La infraestructura empleada corresponde a un clúster MongoDB Atlas M0, configurado en AWS, región São Paulo (sa-east-1), con MongoDB 8.0. La base de trabajo se denomina foodstore_tpi y utiliza tres colecciones: clientes, productos y pedidos. La persistencia se verificó mediante operaciones realizadas desde la aplicación Node.js y consultas en Atlas.

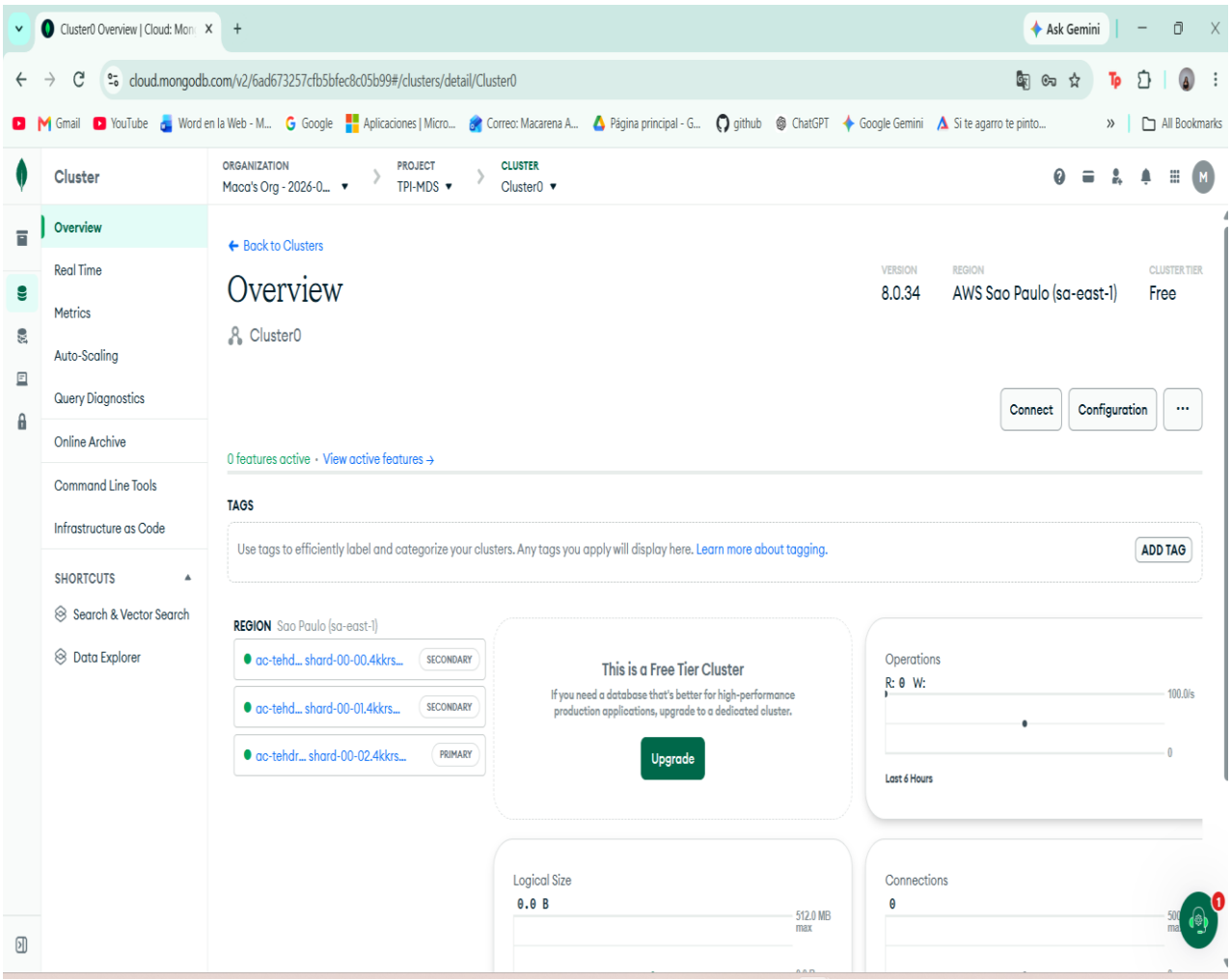


Figura 1. Clúster configurado para la persistencia en la nube.

✓ Database Access y Network Access

Para la conexión con MongoDB Atlas se configuró el usuario de base de datos app_foodstore y las reglas de acceso de red necesarias para el entorno de desarrollo y pruebas. La aplicación utiliza la cadena de conexión definida en un archivo de configuración local, evitando incorporar las credenciales directamente al código fuente. Esta configuración permitió establecer la comunicación entre Node.js y MongoDB Atlas, ejecutar las operaciones CRUD y verificar la persistencia de la información en las tres colecciones del proyecto.

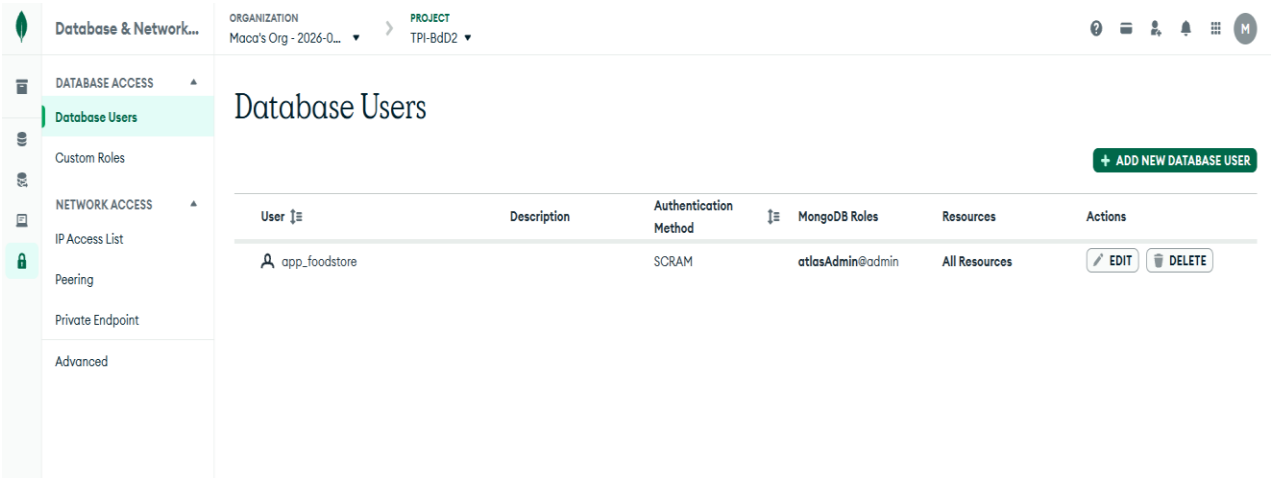


Figura 2. Cuenta de acceso configurada en Atlas.

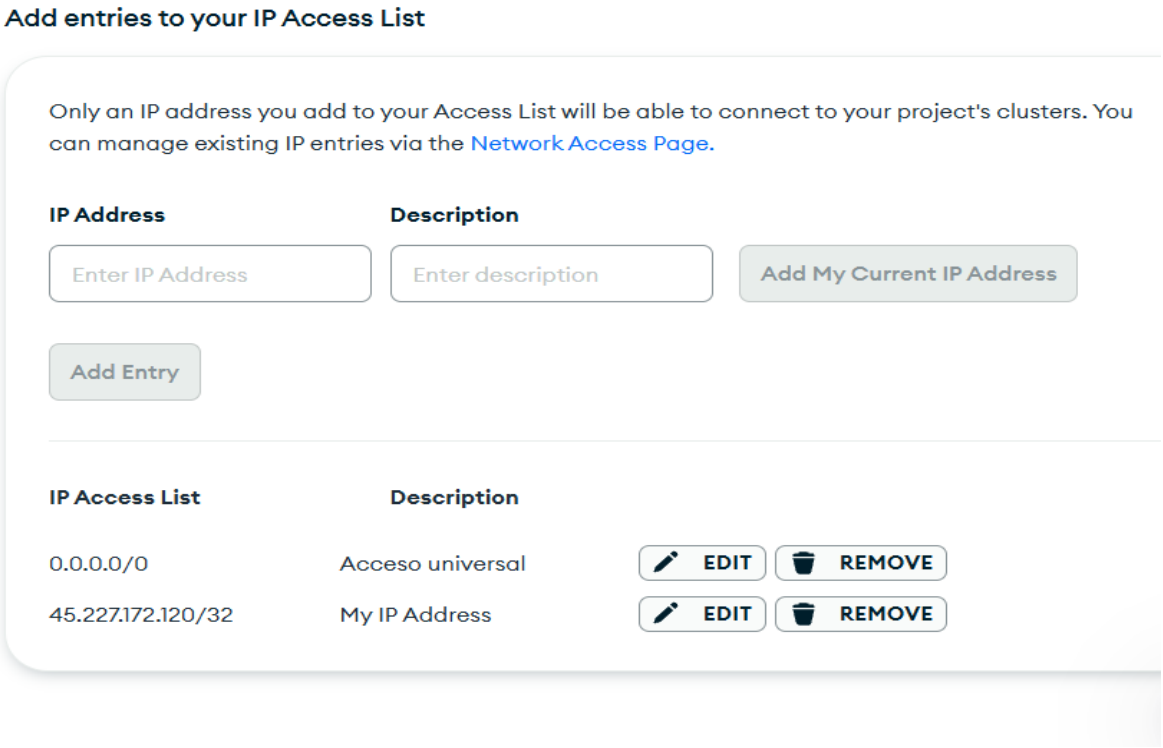


Figura 3. Network Access utilizado durante las pruebas.

La conexión entre la aplicación Food Store y MongoDB Atlas se realiza mediante el controlador oficial de MongoDB para Node.js. Para administrar los parámetros de conexión se utilizan las variables de entorno MONGODB_URI y MONGODB_DB, cargadas mediante la biblioteca dotenv.

Esta configuración permite establecer la comunicación con el clúster y acceder a la base de datos foodstore_tpi, donde se almacenan las colecciones clientes, productos y pedidos. De esta manera, la aplicación puede ejecutar operaciones de consulta, inserción y actualización de documentos, garantizando la persistencia de la información en la base de datos alojada en la nube.

✓ Colecciones e índice único

La existencia de las tres colecciones se acredita mediante capturas de Atlas incorporadas inmediatamente después del índice único y también referenciadas en el material complementario:

- Colección clientes.
- Colección productos.
- Colección pedidos.

En `clientes` se definió el índice único `email_1`, que impide duplicar el campo `email`, incluidos los registros con `activo: false`. El índice fue comprobado también después de la restauración.

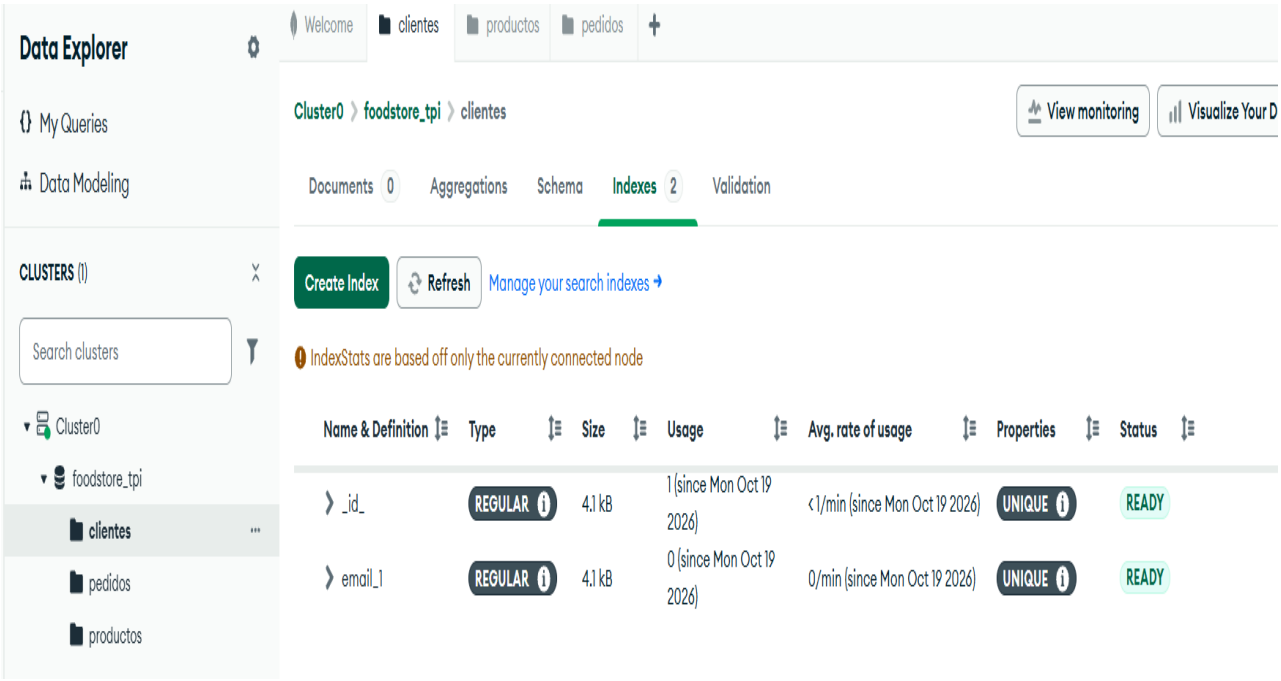


Figura 4. Restricción de unicidad aplicada en la base de datos.

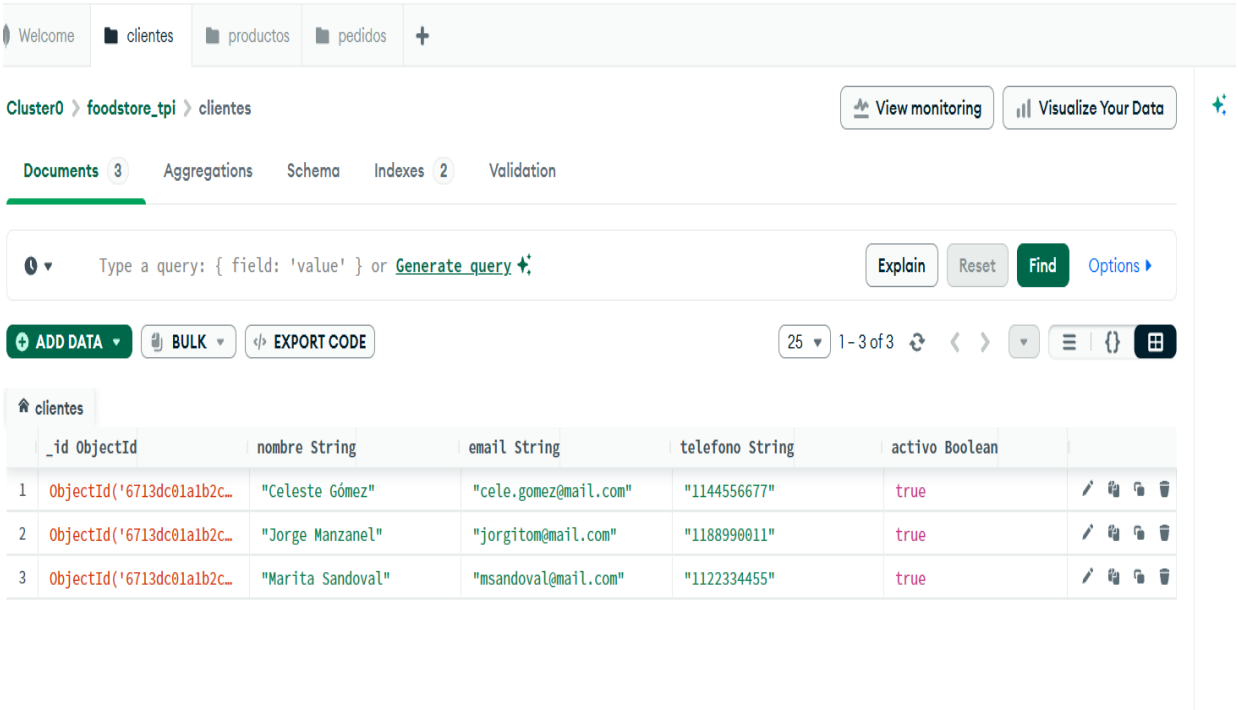


FIGURA 04A · Colección clientes — documentos almacenados en Atlas

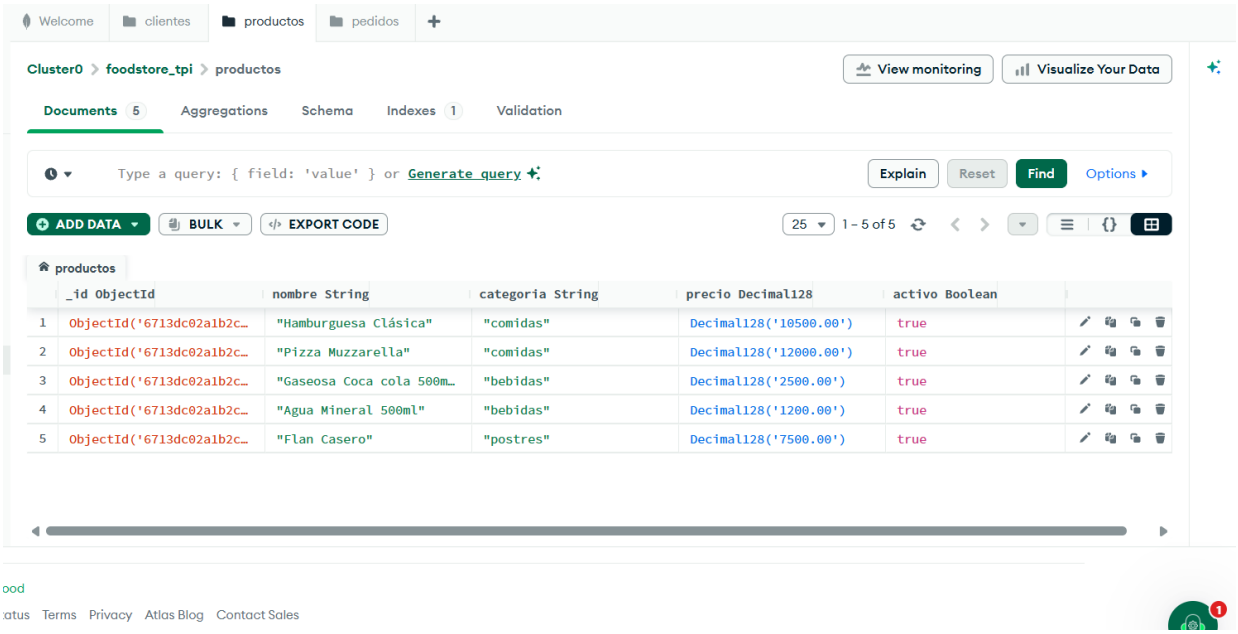


FIGURA 04B · Colección productos — catálogo almacenado en Atlas

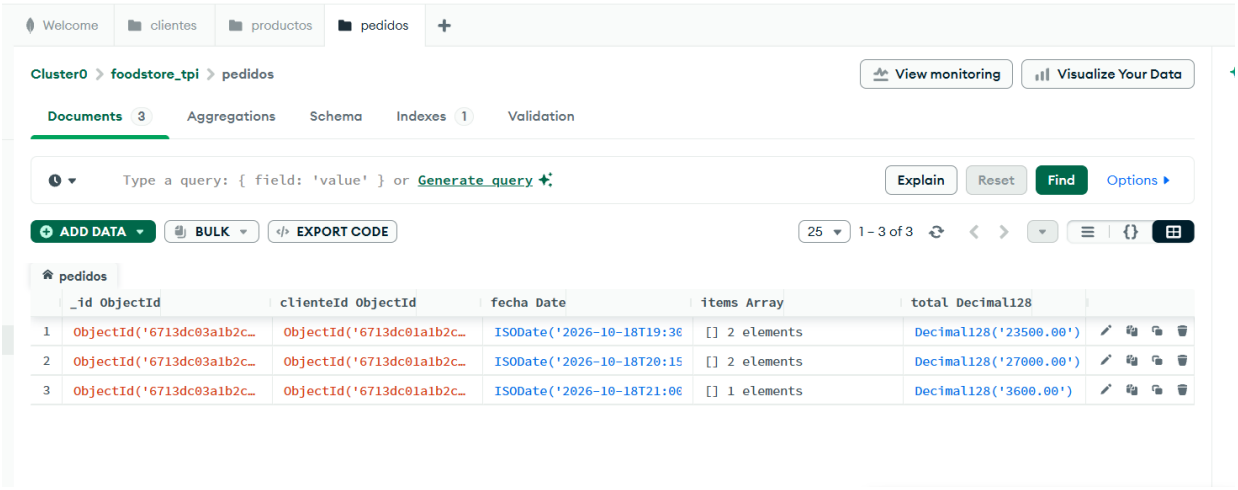


FIGURA 04C · Colección pedidos — transacciones almacenadas en Atlas

6. Modelo de datos documental

✓ Esquema de las tres colecciones

Colección	Campos principales	Tipos destacados
clientes	_id, nombre, email, telefono, activo	ObjectId, String, Boolean
productos	_id, nombre, categoria, precio, activo	ObjectId, String, Decimal128, Boolean
pedidos	_id, clienteId, fecha, items, total, estado, activo	ObjectId, Date, Array, Decimal128, Boolean
pedidos.items[]	productoId, nombreProducto, precioUnitario, cantidad, subtotal	ObjectId, String, Decimal128, Int32

Para el diseño del modelo documental de Food Store se seleccionaron los tipos BSON de acuerdo con las características de cada dato. Los números telefónicos se almacenan como String, ya que representan información de contacto y no requieren operaciones matemáticas. Los precios, subtotales y totales se almacenan mediante Decimal128, lo que permite representar importes monetarios con precisión decimal y evitar los inconvenientes asociados al uso de números de punto flotante. Por su parte, las cantidades de productos incluidas en los pedidos se representan mediante Int32, dado que corresponden a valores enteros positivos.

Esta organización permite mantener una estructura documental coherente, facilitar el procesamiento de la información y asegurar que cada campo utilice un tipo de dato apropiado para las operaciones realizadas por la aplicación.

✓ Ejemplo de cliente en Extended JSON

```
{
  "_id": { "$oid": "6713dc01a1b2c3d4e5f60001" },
  "nombre": "Cliente Ejemplo",
  "email": "cliente.ejemplo@example.com",
  "telefono": "1100000000",
  "activo": true
}
```

✓ Ejemplo de producto en Extended JSON

```
{
  "_id": { "$oid": "6713dc02a1b2c3d4e5f60010" },
  "nombre": "Hamburguesa Clásica",
  "categoria": "comidas",
  "precio": { "$numberDecimal": "10500.00" },
  "activo": true
}
```

✓ Ejemplo de pedido en Extended JSON

```
{
  "_id": { "$oid": "6713dc03a1b2c3d4e5f60020" },
  "clienteId": { "$oid": "6713dc01a1b2c3d4e5f60001" },
  "fecha": { "$date": "2026-10-18T19:30:00.000Z" },
  "items": [
    {
      "productoId": { "$oid": "6713dc02a1b2c3d4e5f60010" },
      "nombreProducto": "Hamburguesa Clásica",
      "precioUnitario": { "$numberDecimal": "10500.00" },
      "cantidad": { "$numberInt": "2" },
      "subtotal": { "$numberDecimal": "21000.00" }
    }
  ],
  "total": { "$numberDecimal": "21000.00" },
  "estado": "pendiente",
  "activo": true
}
```

Los ejemplos presentados permiten visualizar la estructura documental utilizada en Food Store y la representación de los distintos tipos de datos BSON mediante el formato Extended JSON.

Se incluyen documentos correspondientes a las tres colecciones principales: clientes, productos y pedidos. En ellos se pueden observar los identificadores ObjectId, las referencias entre documentos, los valores monetarios almacenados mediante Decimal128 y el detalle embebido de los productos incluidos en cada pedido. Estos ejemplos complementan la explicación del modelo NoSQL y permiten comprender cómo se organizan y relacionan los datos dentro de MongoDB.

✓ Justificación del linking

Los pedidos referencian el cliente mediante `clienteId` y los productos mediante `items[].productoId`. Se utiliza **linking** porque clientes y productos tienen vida propia y pueden aparecer en múltiples pedidos. El modelo evita copiar todos los datos del cliente y del catálogo en cada transacción.

La aplicación verifica la existencia y condición activa de los documentos referenciados antes de crear el pedido. MongoDB no impone estas referencias como claves foráneas; las verificaciones previas y la inserción del pedido no forman una transacción conjunta, por lo que no se declara una garantía frente a modificaciones concurrentes.

✓ Justificación del embedding

Los ítems se almacenan como documentos **embebidos** en `pedidos.items` porque conforman el detalle de una transacción y se consultan junto con ella. Se incluyen una referencia al producto y una copia de `nombreProducto` y `precioUnitario`.

Esta duplicación controlada mantiene los datos del momento de compra: un cambio posterior del precio o la desactivación del producto no modifica el valor histórico del pedido. Esto se verificó en las pruebas `PED-08` y `PED-09`.

El detalle ampliado del diseño se documenta en Modelo y decisiones de diseño.

7. Operaciones CRUD y reglas de negocio

✓ Clientes

- **Crear:** registra un cliente con `activo: true` y valida campos obligatorios.
- **Leer:** lista y busca únicamente clientes activos.
- **Modificar:** actualiza campos permitidos del documento.
- **Desactivar:** cambia `activo` a `false` sin eliminar el documento.
- **Unicidad:** normaliza el correo a minúsculas y elimina espacios externos; el índice único impide reutilizarlo, incluso si el titular anterior fue desactivado.

Se verificaron alta y persistencia (`CLI-01`), búsqueda (`CLI-02`), modificación (`CLI-03`), rechazo de duplicados (`CLI-05`), baja lógica (`CLI-08`) y tratamiento de correo de cliente inactivo (`CLI-09`).

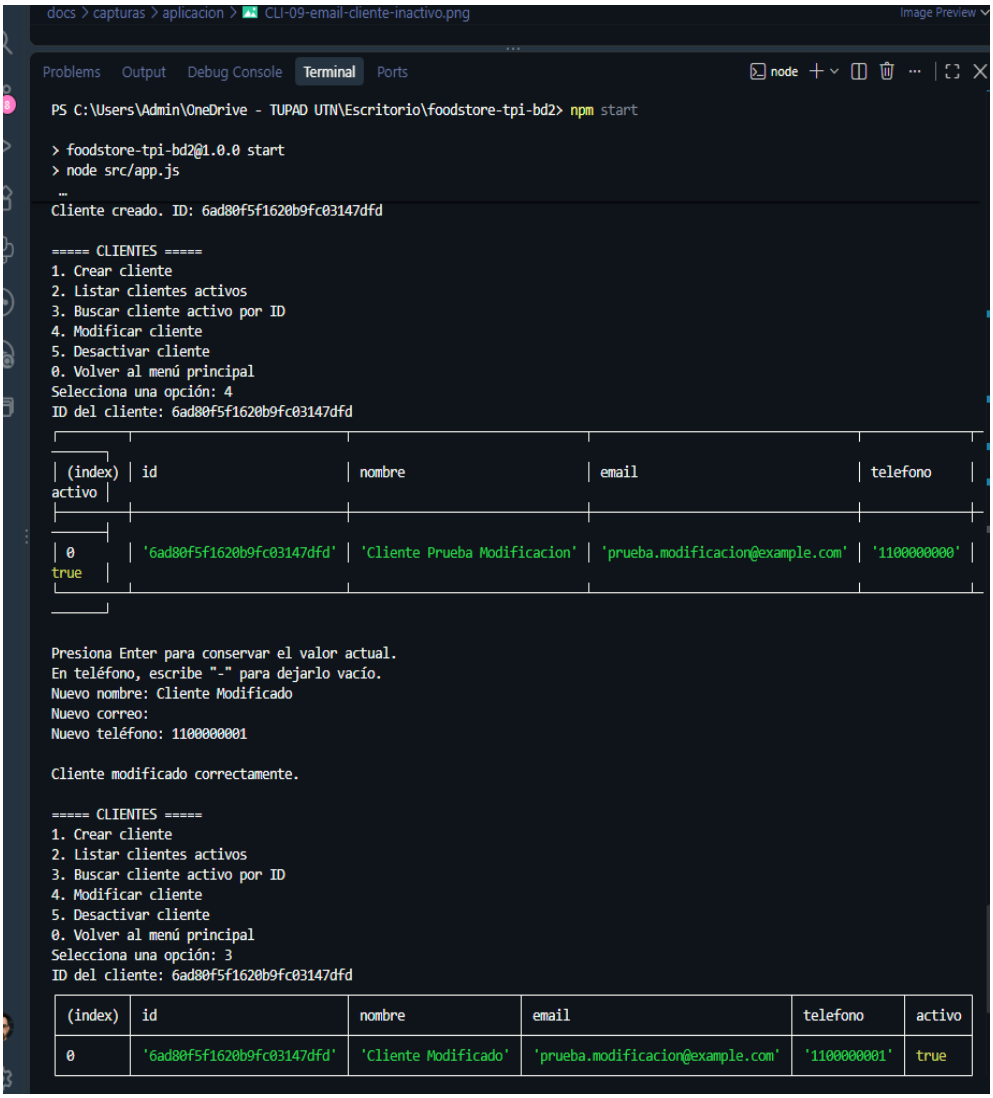


Figura 5. Evidencia de actualización de clientes.

✓ Productos

- **Crear y leer:** registra y consulta productos activos.
- **Modificar:** permite actualizar los campos admitidos, incluido el precio.
- **Desactivar:** establece `activo: false` y lo excluye de las búsquedas de activos.
- **Validar:** admite categorías `comidas`, `bebidas` o `postres`, y precios positivos con hasta dos decimales; acepta coma o punto decimal como separador de entrada.

Las capturas `PRO-01` a `PRO-07` muestran las operaciones principales, el rechazo de categorías o importes inválidos y la baja lógica.



Figura 6. Cancelación y confirmación de baja lógica de un producto.

✓ Pedidos

La aplicación comprueba que el cliente y los productos elegidos existan y se encuentren activos. Al crear un pedido calcula el subtotal de cada ítem con `decimal.js`, almacena la cantidad como `Int32` y registra subtotal y total como `Decimal128`.

Reglas principales:

- Los pedidos nuevos comienzan en `estado: "pendiente"` y `activo: true`.
- Los estados permitidos después de `pendiente` son `entregado` y `cancelado`; no se permiten cambios posteriores.
- La baja lógica modifica únicamente el indicador `activo` y conserva el pedido con su detalle.
- `estado: "cancelado"` y `activo: false` representan conceptos distintos.

La prueba `PED-02` utilizó dos unidades de ARS 1.250,50 y una unidad de ARS 750,25: ARS 2.501,00 + ARS 750,25 = **ARS 3.251,25**.

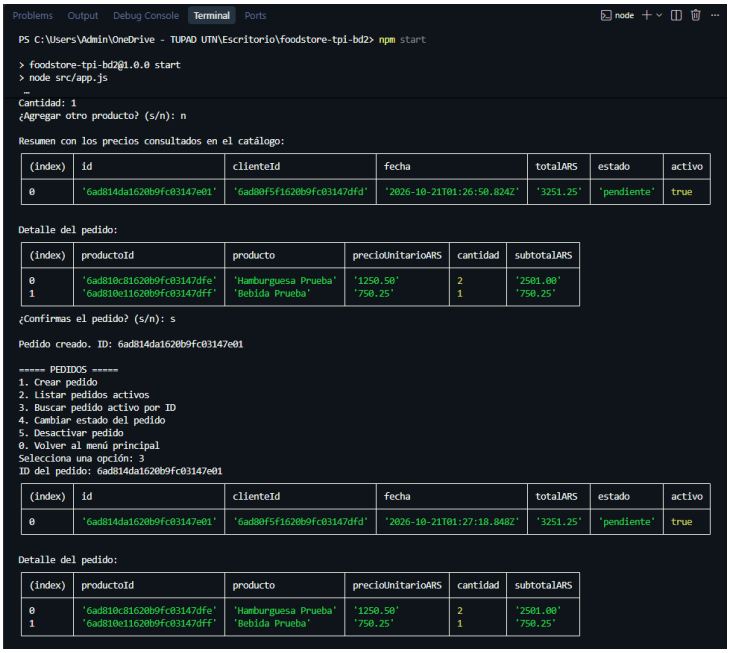


Figura 7. Pedido de prueba con subtotal y total.

En `PED-13`, se comprobó posteriormente que el pedido desactivado seguía almacenado en la base principal con `activo: false` y **no aparecía** en el listado de pedidos activos de Food Store. La base de recuperación conservaba una versión anterior en la que figuraba como activo.

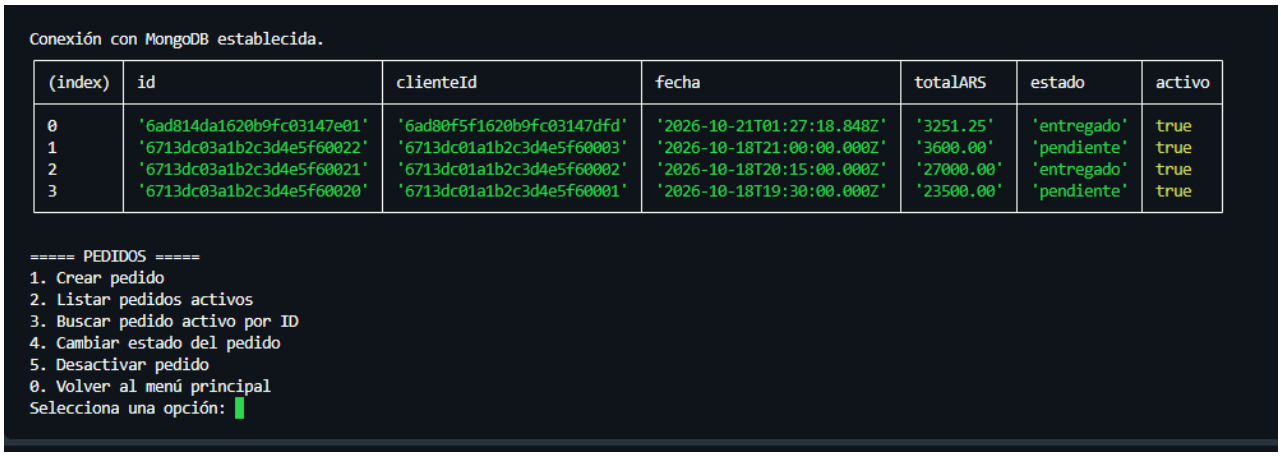


Figura 8. Consulta de pedidos activos después de la baja lógica.

✓ Comprobación adicional de integridad

Durante la auditoría se detectó un error en un dato de ejemplo: el pedido terminado en `f60021` tenía una Pizza Muzzarella con `precioUnitario` ARS 1.200,00 y subtotal ARS 12.000,00 para cantidad 1. Después de contrastar el dato con el catálogo y el subtotal, se corrigió controladamente el precio unitario a **ARS 12.000,00**, sin alterar el total del pedido.

A continuación se revisaron los cálculos de todos los pedidos de la base principal con una consulta de solo lectura y aritmética exacta en centavos:

Indicador	Resultado
Pedidos analizados	5
Pedidos aritméticamente correctos	5
Inconsistencias detectadas tras la corrección	0

Esta comprobación valida los importes de los cinco pedidos examinados, no todas las reglas de negocio ni la concurrencia entre operaciones.

8. Mecanismo de respaldo

✓ Script y funcionamiento

El archivo `scripts/backup.ps1` lee `MONGODB_URI` y `MONGODB_DB` desde el `.env` local, verifica la presencia de `mongodump` y ejecuta el respaldo de toda la base. No filtra por `activo`, por lo que incluye documentos activos e inactivos.

El script utiliza rutas relativas al directorio del proyecto y guarda cada nueva ejecución en:

```
resguardos_tpi/
  AAAA-MM-DD/
    HH-mm-ss/
      foodstore_tpi/
        clientes.bson
        clientes.metadata.json
        productos.bson
        productos.metadata.json
        pedidos.bson
        pedidos.metadata.json
        prelude.json
      resultado_backup.txt
```

El respaldo original del 20/10/2026 utiliza una carpeta por fecha sin la subcarpeta de hora, porque se generó antes de la mejora. Si `mongodump` falla, el script devuelve un código distinto de cero y la copia parcial no debe considerarse un respaldo válido. Las ejecuciones correctas quedan registradas en `resultado_backup.txt`.

✓ Ejecuciones documentadas

Fecha y hora local	Duración	Datos informados
20/10/2026, 22:48	8,21 s	5 clientes, 8 productos, 5 pedidos; 3 BSON; código 0
21/10/2026, 00:50	1,93 s	18 documentos; 3 BSON; código 0
21/10/2026, 01:22	2,02 s	18 documentos; 3 BSON; código 0; posterior a la corrección del dato

El primer respaldo ocupó **3.904 bytes (3,81 KB)** en su carpeta de datos, sin contar `resultado_backup.txt`. Estas cifras corresponden al entorno de prueba y pueden variar en futuras ejecuciones.



Figura 9. Ejecución exitosa del script de respaldo.

9. Restauración y comprobación de recuperación

Se realizó una prueba de restauración utilizando el respaldo del **20/10/2026**, mediante mongorestore. Los datos se recuperaron en la base foodstore_tpi_restore_test, sin modificar la base principal.

Procedimiento realizado:

1. Verificación de los archivos BSON y sus metadatos.
2. Validación previa mediante mongorestore --dryRun.
3. Restauración de las tres colecciones en la base de pruebas.
4. Comprobación de documentos e índices mediante mongosh.
5. Verificación de consultas desde la aplicación Food Store.

Resultado: se recuperaron correctamente **18 documentos en 2,43 segundos**, sin errores. También se verificó el acceso a la información desde la aplicación.

Indicador de recuperación	Resultado
Documentos restaurados	18
Fallos reportados	0
Distribución	5 clientes, 8 productos, 5 pedidos
Índice de clientes	email_1 con unique: true, verificado
Duración de mongorestore	2,43 segundos
Código de salida	0
Consultas desde la aplicación	4 clientes activos, 6 productos activos y 5 pedidos activos

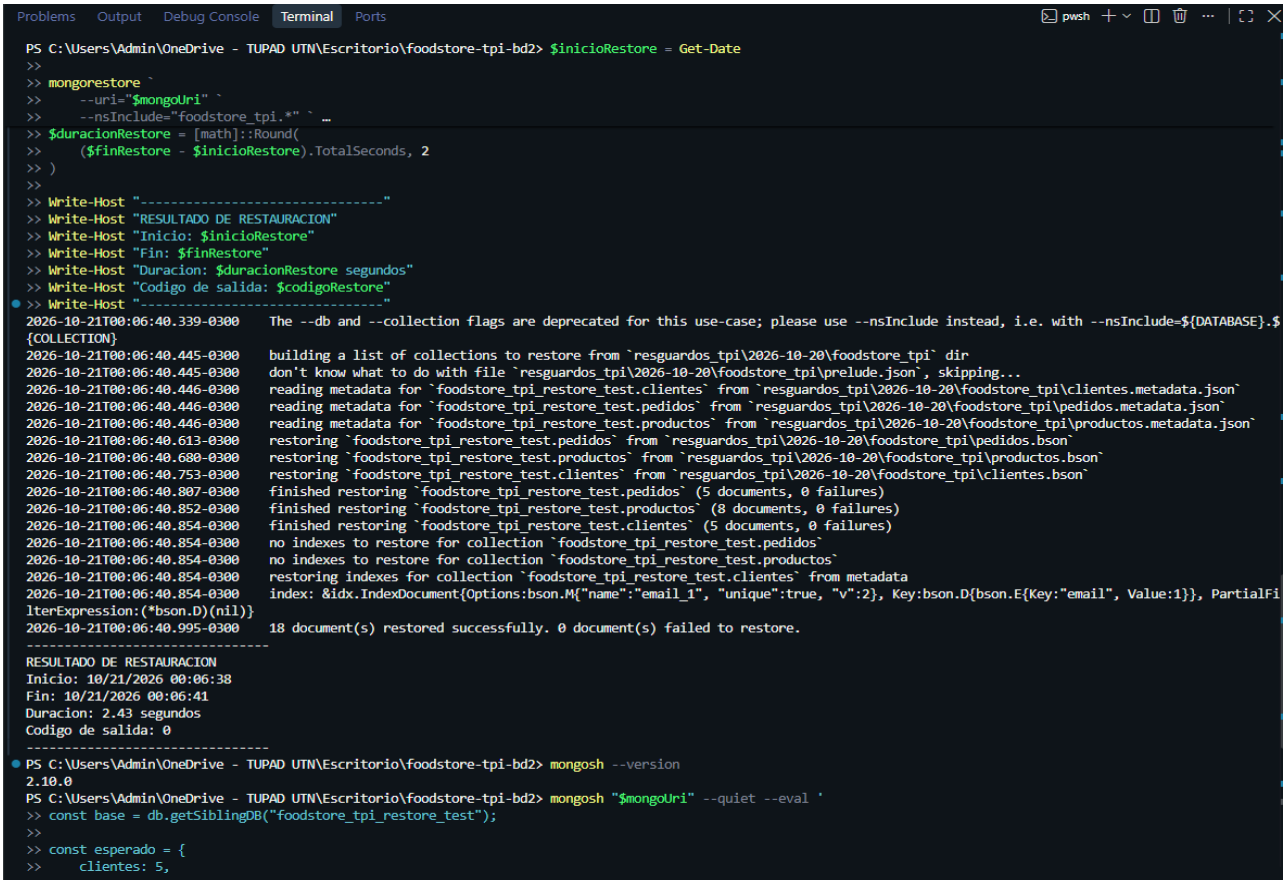


Figura 10. Restauración real de los 18 documentos en la base de pruebas.

```
PS C:\Users\Admin\OneDrive - TUPAD UTN\Escritorio\foodstore-tpi-bd2> mongosh "$mongoUri" --quiet --eval '
>> const base = db.getSiblingDB("foodstore_tpi_restore_test");
>>
>> const esperado = {
>>     clientes: 5,
>>     pedidos: 5,
>>     productos: 8
>> };
>>
>> for (const [coleccion, cantidadEsperada] of Object.entries(esperado)) {
>>     const cantidadReal = base.getCollection(coleccion).countDocuments({});
>>     const estado = cantidadReal === cantidadEsperada ? "OK" : "REVISAR";
>>     print(`${coleccion}: ${cantidadReal} documentos - ${estado}`);
>> }
>>
>> const indice = base.clientes.getIndexes().find(i => i.name === "email_1");
>> print("Indice email_1 unico: " + (indice?.unique === true ? "OK" : "REVISAR"));
>> '
clientes: 5 documentos - OK
pedidos: 5 documentos - OK
productos: 8 documentos - OK
Indice email_1 unico: OK
PS C:\Users\Admin\OneDrive - TUPAD UTN\Escritorio\foodstore-tpi-bd2>
```

Figura 11. Consulta de cantidades e índice único recuperado.

La copia utilizada en esta restauración es anterior a la corrección de Pizza Muzzarella y a la baja lógica posterior del pedido `PED-13`. Por ese motivo, una base restaurada puede presentar valores distintos de la base principal actual. **No se ejecutó una restauración independiente del backup posterior de las 01:22.**

10. Objetivos de recuperación RTO y RPO

✓ RPO

El **Recovery Point Objective (RPO)** establece la pérdida máxima tolerable de datos, expresada en tiempo. Para Food Store se propone un **RPO de 24 horas**.

Aunque el script se ejecutó exitosamente en tres ocasiones, su uso actual es **manual**. No existe una tarea periódica supervisada que garantice la existencia de una copia válida con una antigüedad máxima de 24 horas ante cualquier incidente. Por lo tanto, **el objetivo RPO no se considera garantizado**.

✓ RTO

El **Recovery Time Objective (RTO)** representa el tiempo objetivo para restablecer el funcionamiento después de una interrupción. Para el entorno académico se propone un **RTO de 60 minutos**.

Los **2,43 segundos** corresponden al tiempo observado de la herramienta `mongorestore`. No incluyen detección del incidente, localización de la copia, preparación de accesos, verificación de los datos y comprobación final del funcionamiento en un único proceso cronometrado. Por eso **no se afirma que el RTO integral de 60 minutos esté validado**.

El desarrollo de estos conceptos, procedimientos y limitaciones está en Objetivos de recuperación RTO/RPO.

11. Resultados de las pruebas y evidencias

Área	Resultado comprobado
Conexión	Atlas accesible desde la aplicación; consultas y operaciones persistentes
Clientes	Alta, lectura, modificación, correos duplicados y baja lógica
Productos	Alta, lectura, modificación, validaciones y baja lógica
Pedidos	Creación, importes, cambios de estado, historial y baja lógica
Integridad	5 pedidos revisados, 0 inconsistencias aritméticas después de corregir el dato
Backup	3 ejecuciones exitosas, con datos de las tres colecciones
Recuperación	18 documentos restaurados y consultados desde Food Store

El plan de pruebas describe los casos y sus criterios; el registro de ejecución detalla qué fue observado y con qué evidencia. No todos los casos complementarios del catálogo inicial fueron ejecutados, y no se presentan como aprobados.

12. Limitaciones y propuestas de mejora

Entre las mejoras identificadas para un entorno real se encuentran:

- Restringir Network Access a los orígenes necesarios y asignar al usuario de aplicación permisos acordes con sus funciones.
- Mantener credenciales privadas y excluir `.env`, respaldos locales, dependencias instaladas e historial Git del ZIP público.
- Programar y supervisar respaldos si se pretende sostener el RPO propuesto; mantener copias independientes y una política de retención.
- Realizar pruebas integrales de recuperación con medición del RTO completo, incluidas comprobaciones de la copia más reciente.
- Incorporar pruebas automatizadas y controles adicionales frente a cambios concurrentes entre validación e inserción de pedidos.

Son **mejoras propuestas**, no funcionalidades que se declaren implementadas o ensayadas.

13. Conclusión

Food Store permitió aplicar de manera práctica los conceptos de modelado documental, persistencia Cloud y comunicación cliente-servidor. La separación de clientes, productos y pedidos mediante referencias mantuvo entidades independientes, mientras que el uso de documentos embebidos en `items` permitió conservar el detalle histórico de cada compra y consultarlo como parte del pedido.

La experiencia de desarrollo también mostró la importancia de validar los datos en varios niveles. La aplicación aplica las reglas de negocio, el índice único protege el correo a nivel de base, y las pruebas sobre documentos reales permitieron identificar y corregir una inconsistencia en el precio unitario de un pedido. La baja lógica conservó los documentos sin exponerlos en los listados habituales de activos.

En cuanto a la comunicación cliente-servidor, las pruebas confirmaron que la aplicación Node.js envía operaciones por medio del driver y recibe respuestas de MongoDB Atlas, con datos que permanecen disponibles después del cierre del programa. Las pruebas de respaldo y restauración reforzaron esa verificación al recuperar 18 documentos en otra base y consultarlos nuevamente desde Food Store.

El trabajo requirió coordinar el modelo, la configuración de Atlas, la implementación de los módulos y la documentación de evidencias. El resultado es un proyecto académico funcional dentro del alcance definido. Se identificaron además mejoras necesarias para un uso operativo, especialmente en permisos, automatización y monitoreo de respaldos, recuperación integral y pruebas más extensas. Estas limitaciones se describen explícitamente, sin presentar objetivos o funcionalidades pendientes como si estuvieran comprobados.

14. Material complementario

Como complemento del presente informe, el proyecto Food Store dispone de un repositorio en GitHub que contiene el código fuente de la aplicación, el modelo documental NoSQL, los archivos JSON de ejemplo, el script de respaldo y la documentación de las pruebas realizadas.

El repositorio permite consultar la implementación desarrollada y las evidencias de funcionamiento de MongoDB Atlas, las operaciones CRUD y los procedimientos de respaldo y recuperación.

Repositorio del proyecto: https://github.com/CastoGil/foodstore-tpi-bd2

ANEXO I · EVIDENCIAS COMPLEMENTARIAS

Esta galería reúne capturas complementarias de navegación, validaciones, operaciones CRUD y recuperación. Las imágenes principales de Atlas, baja lógica y restauración ya están incorporadas en los apartados correspondientes del informe.

Aplicación de consola, clientes, productos y pedidos

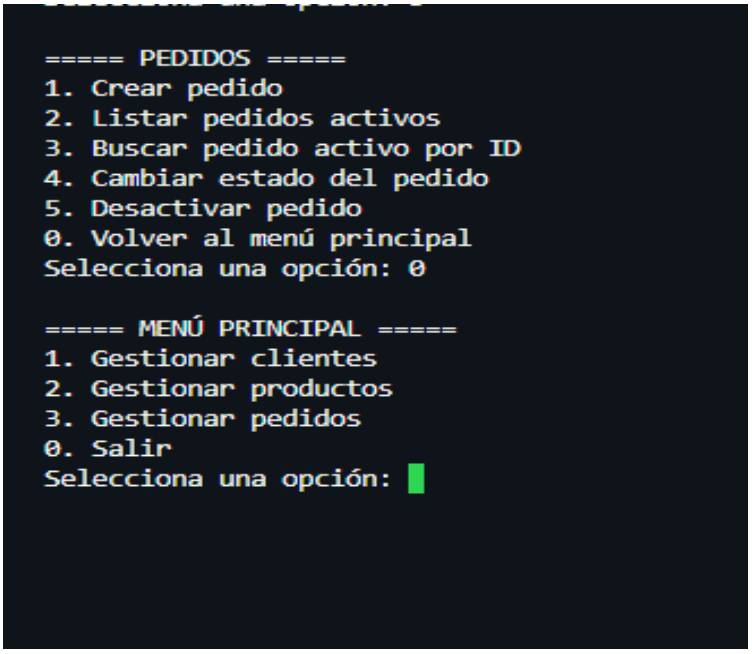


FIGURA 12 · APP-01 — Menú pedidos · docs/capturas/aplicacion/APP-01-menu-pedidos.png

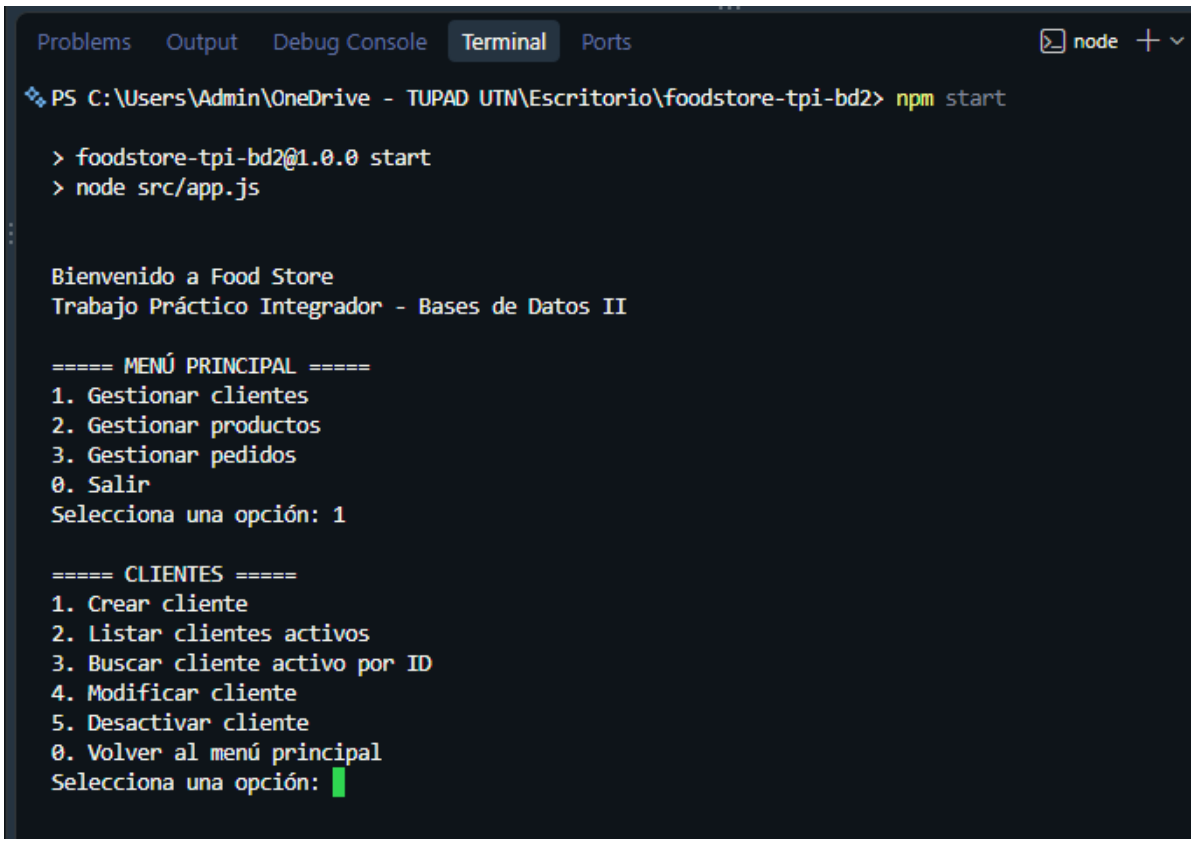


FIGURA 13 · APP-01 — Menú principal · docs/capturas/aplicacion/APP-01-menu-principal.png

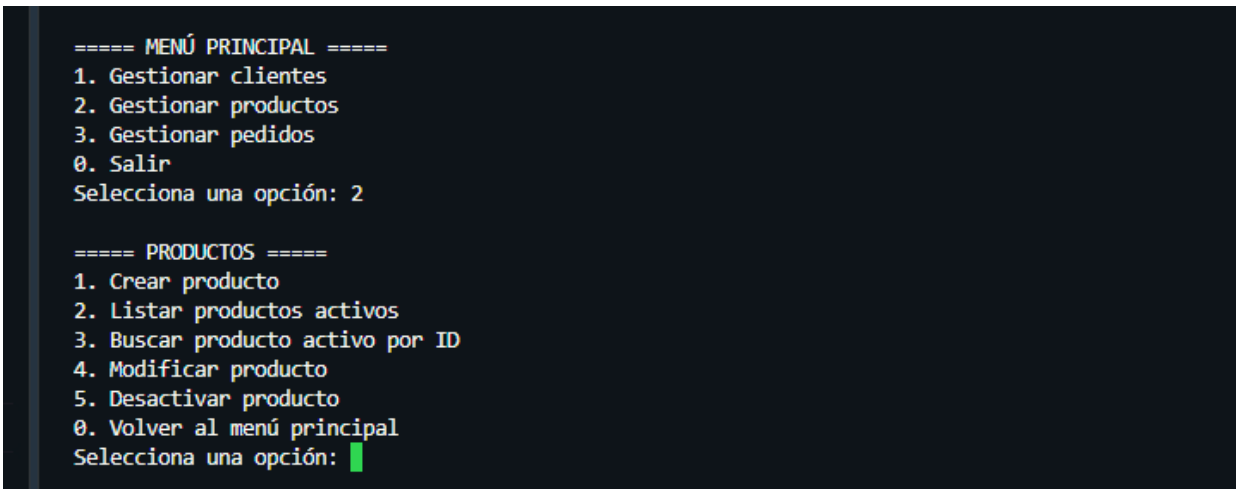


FIGURA 14 · APP-01 — Menú productos · docs/capturas/aplicacion/APP-01-menu-productos.png



FIGURA 15 · APP-01 — Navegación completa · docs/capturas/aplicacion/APP-01-navegacion-completa.png

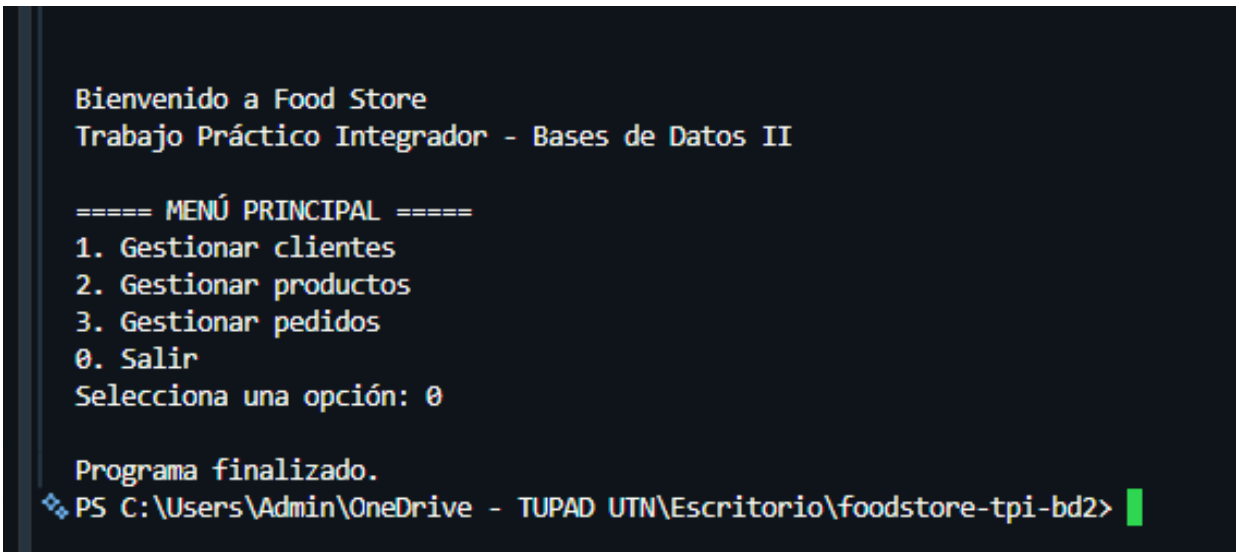


FIGURA 16 · APP-01 — Salida del programa · docs/capturas/aplicacion/APP-01-salida-programa.png

```
===== CLIENTES =====
1. Crear cliente
2. Listar clientes activos
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción: 2

Conexión con MongoDB establecida.
```

(index)	id	nombre	email	telefono	activo
0	'6713dc01a1b2c3d4e5f60001'	'Celeste Gómez'	'cele.gomez@mail.com'	'1144556677'	true
1	'6713dc01a1b2c3d4e5f60002'	'Jorge Manzanel'	'jorgitom@mail.com'	'1188990011'	true
2	'6713dc01a1b2c3d4e5f60003'	'Marita Sandoval'	'msandoval@mail.com'	'1122334455'	true

FIGURA 17 · APP-02 — Clientes en Atlas · docs/capturas/aplicacion/APP-02-clientes-atlas.png

```
PS C:\Users\Admin\OneDrive - TUPAD UTN\Escritorio\foodstore-tpi-bd2> npm start

  > foodstore-tpi-bd2@1.0.0 start
  > node src/app.js


  Bienvenido a Food Store
  Trabajo Práctico Integrador - Bases de Datos II

  ===== MENÚ PRINCIPAL =====
  1. Gestionar clientes
  2. Gestionar productos
  3. Gestionar pedidos
  0. Salir
  Selecciona una opción: 9

  Opción inválida. Ingresa 1, 2, 3 o 0.

  ===== MENÚ PRINCIPAL =====
  1. Gestionar clientes
  2. Gestionar productos
  3. Gestionar pedidos
  0. Salir
  Selecciona una opción:
```

FIGURA 18 · APP-02 — Opción inválida · docs/capturas/aplicacion/APP-02-opcion-invalida.png

```
Problems  Output  Debug Console  Terminal  Ports                                  node

PS C:\Users\Admin\OneDrive - TUPAD UTN\Escritorio\foodstore-tpi-bd2> npm start

===== MENÚ PRINCIPAL =====
1. Gestionar clientes
2. Gestionar productos
3. Gestionar pedidos
0. Salir
Selecciona una opción: 2

===== PRODUCTOS =====
1. Crear producto
2. Listar productos activos
3. Buscar producto activo por ID
4. Modificar producto
5. Desactivar producto
0. Volver al menú principal
Selecciona una opción: 2

No se pudo completar la operación: Falta configurar MONGODB_URI en el archivo .env.

===== PRODUCTOS =====
1. Crear producto
2. Listar productos activos
3. Buscar producto activo por ID
4. Modificar producto
5. Desactivar producto
0. Volver al menú principal
Selecciona una opción:
```

FIGURA 19 · APP-03 — URI no configurada · docs/capturas/aplicacion/APP-03-sin-uri.png

```
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción: 2

Conexión con MongoDB establecida.
```

(index)	id	nombre	email	telefono	activo
0	'6713dc01a1b2c3d4e5f60001'	'Celeste Gómez'	'cele.gomez@mail.com'	'1144556677'	true
1	'6713dc01a1b2c3d4e5f60002'	'Jorge Manzanel'	'jorgitom@mail.com'	'1188990011'	true
2	'6713dc01a1b2c3d4e5f60003'	'Marita Sandoval'	'msandoval@mail.com'	'1122334455'	true

```
===== CLIENTES =====
1. Crear cliente
2. Listar clientes activos
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción: 1
Nombre completo: Cliente Prueba Ricardo
Correo electrónico: prueba.ricardo@example.com
Teléfono (opcional): 1100000000

Cliente creado. ID: 6ad80aae1620b9fc03147dfa

===== CLIENTES =====
1. Crear cliente
2. Listar clientes activos
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción: 2
```

(index)	id	nombre	email	telefono	activo
0	'6713dc01a1b2c3d4e5f60001'	'Celeste Gómez'	'cele.gomez@mail.com'	'1144556677'	true
1	'6ad80aae1620b9fc03147dfa'	'Cliente Prueba Ricardo'	'prueba.ricardo@example.com'	'1100000000'	true
2	'6713dc01a1b2c3d4e5f60002'	'Jorge Manzanel'	'jorgitom@mail.com'	'1188990011'	true
3	'6713dc01a1b2c3d4e5f60003'	'Marita Sandoval'	'msandoval@mail.com'	'1122334455'	true

FIGURA 20 · CLI-01 — Cliente persistido · docs/capturas/aplicacion/CLI-01-cliente-persistido.png

```
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción: 3
ID del cliente: 6ad80aae1620b9fc03147dfa
```

(index)	id	nombre	email	telefono	activo
0	'6ad80aae1620b9fc03147dfa'	'Cliente Prueba Ricardo'	'prueba.ricardo@example.com'	'1100000000'	true

```
===== CLIENTES =====
1. Crear cliente
2. Listar clientes activos
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción:
```

FIGURA 21 · CLI-02 — Búsqueda de cliente · docs/capturas/aplicacion/CLI-02-busqueda-cliente.png

```
1. Crear cliente
2. Listar clientes activos
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción: 1
Nombre completo: Ricardo
Correo electrónico: prueba.ricardo@example.com
Teléfono (opcional): 1100000001

Hay un correo duplicado. Debe ser único, incluso entre clientes desactivados. Si ocurrió al iniciar, revisa los datos existentes antes de crear el índice.

===== CLIENTES =====
1. Crear cliente
2. Listar clientes activos
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción:
```

FIGURA 22 · CLI-05 — Correo electrónico duplicado · docs/capturas/aplicacion/CLI-05-email-duplicado.png

```
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción: 5
ID del cliente: 6ad80aae1620b9fc03147dfa
```

(index)	id	nombre	email	telefono	activo
0	'6ad80aae1620b9fc03147dfa'	'Cliente Prueba Ricardo'	'prueba.ricardo@example.com'	'1100000000'	true

```
¿Confirmas la baja lógica? (s/n): s

Cliente desactivado. El documento permanece en la base.

===== CLIENTES =====
1. Crear cliente
2. Listar clientes activos
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción:
```

FIGURA 23 · CLI-08 — Baja lógica de cliente · docs/capturas/aplicacion/CLI-08-baja-cliente.png

```
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción: 5
ID del cliente: 6ad80aae1620b9fc03147dfa
```

(index)	id	nombre	email	telefono	activo
0	'6ad80aae1620b9fc03147dfa'	'Cliente Prueba Ricardo'	'prueba.ricardo@example.com'	'1100000000'	true

```
¿Confirmas la baja lógica? (s/n): s

Cliente desactivado. El documento permanece en la base.

===== CLIENTES =====
1. Crear cliente
2. Listar clientes activos
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción: 2
```

(index)	id	nombre	email	telefono	activo
0	'6713dc01a1b2c3d4e5f60001'	'Celeste Gómez'	'cele.gomez@mail.com'	'1144556677'	true
1	'6713dc01a1b2c3d4e5f60002'	'Jorge Manzanel'	'jorgitom@mail.com'	'1188990011'	true
2	'6713dc01a1b2c3d4e5f60003'	'Marita Sandoval'	'msandoval@mail.com'	'1122334455'	true

```
===== CLIENTES =====
1. Crear cliente
2. Listar clientes activos
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción: 3
ID del cliente: 6ad80aae1620b9fc03147dfa

No se encontró un cliente activo con ese ID.
```

FIGURA 24· CLI-08 — Listado y búsqueda posteriores a la baja · docs/capturas/aplicacion/CLI-08-listado-tras-baja-y-busqueda.png

```
PS C:\Users\Admin\OneDrive - TUPAD UTN\Escritorio\foodstore-tpi-bd2> npm start

> foodstore-tpi-bd2@1.0.0 start
> node src/app.js
 …
2. Listar clientes activos
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción: 1
Nombre completo: Prueba Correo Inactivo
Correo electrónico: prueba.ricardc@example.com
Teléfono (opcional): 1100000000

Hay un correo duplicado. Debe ser único, incluso entre clientes desactivados. Si ocurrió al iniciar,
 revisa los datos existentes antes de crear el índice.

===== CLIENTES =====
1. Crear cliente
2. Listar clientes activos
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción:
```

FIGURA 25 · CLI-09 — Correo asociado a cliente inactivo · docs/capturas/aplicacion/CLI-09-email-cliente-inactivo.png

```
===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción: 1
ID del cliente: 6ad80f5f1620b9fc03147dfd

Cliente: Cliente Modificado
ID del producto: 6ad810e11620b9fc03147dff
Producto: Bebida Prueba
Precio ARS: 750.25
Cantidad: 1
¿Agregar otro producto? (s/n): n

Resumen con los precios consultados en el catálogo:
```

(index)	id	clienteId	fecha	totalARS	estado	activo
0	'6ad8177af44effa3cac4d7f5'	'6ad80f5f1620b9fc03147dfd'	'2026-10-21T01:38:02.677Z'	'750.25'	'pendiente'	true

Detalle del pedido:

(index)	productoId	producto	precioUnitarioARS	cantidad	subtotalARS
0	'6ad810e11620b9fc03147dff'	'Bebida Prueba'	'750.25'	1	'750.25'

```
¿Confirmas el pedido? (s/n): s

Pedido creado. ID: 6ad8177af44effa3cac4d7f5

===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción:
```

FIGURA 26 · PED-01 — Pedido con un producto · docs/capturas/aplicacion/PED-01-pedido-un-producto.png



```
===== PRODUCTOS =====
1. Crear producto
2. Listar productos activos
3. Buscar producto activo por ID
4. Modificar producto
5. Desactivar producto
0. Volver al menú principal
Selecciona una opción: 0

===== MENÚ PRINCIPAL =====
1. Gestionar clientes
2. Gestionar productos
3. Gestionar pedidos
0. Salir
Selecciona una opción: 3

===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción: 3
ID del pedido: 6ad814da1620b9fc03147e01
```

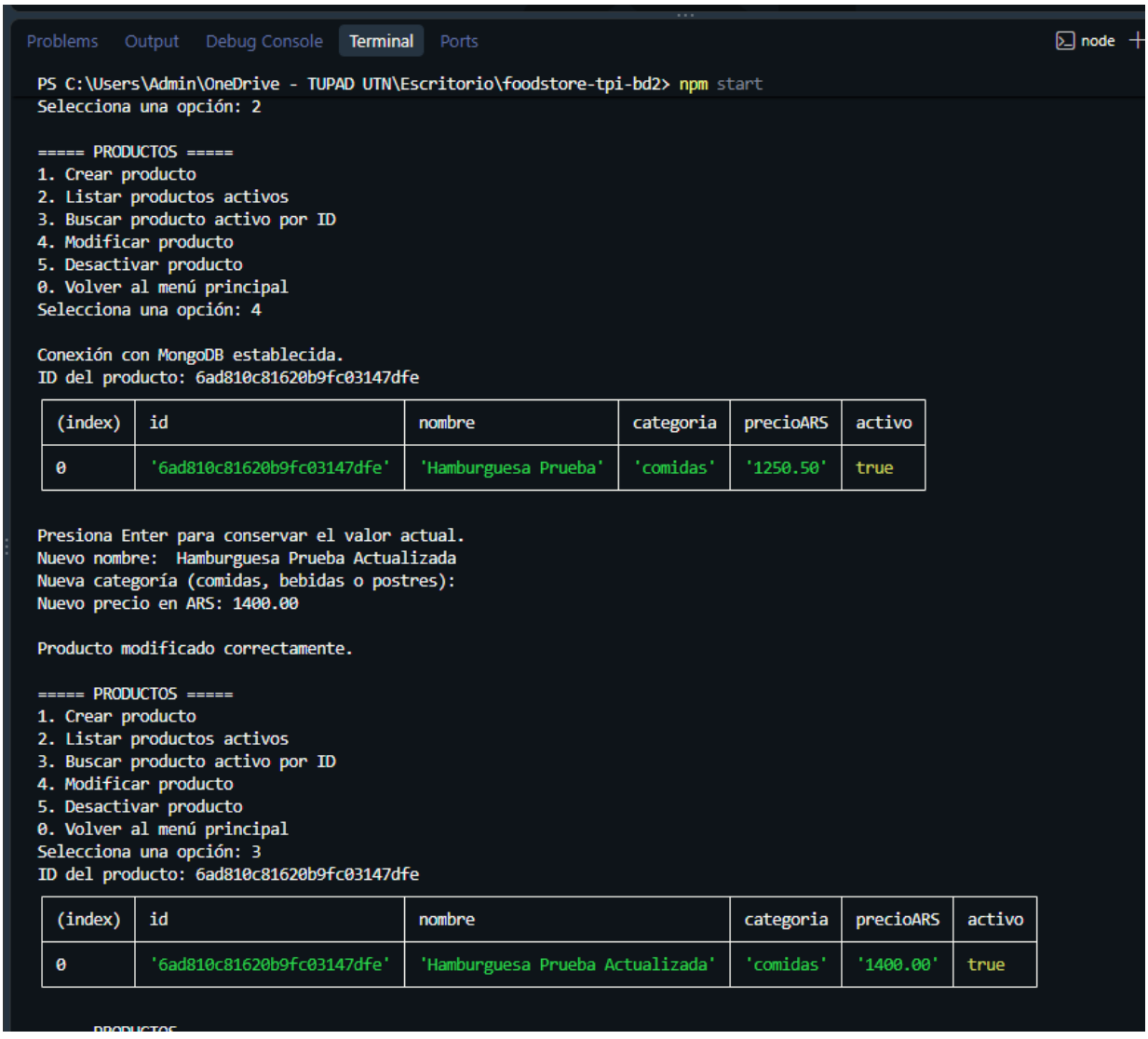
(index)	id	clienteId	fecha	totalARS	estado	activo
0	'6ad814da1620b9fc03147e01'	'6ad80f5f1620b9fc03147dfd'	'2026-10-21T01:27:18.848Z'	'3251.25'	'pendiente'	true

Detalle del pedido:

(index)	productoId	producto	precioUnitarioARS	cantidad	subtotalARS
0	'6ad810c81620b9fc03147dfe'	'Hamburguesa Prueba'	'1250.50'	2	'2501.00'
1	'6ad810e11620b9fc03147dff'	'Bebida Prueba'	'750.25'	1	'750.25'

```
===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
```

FIGURA 27 · PED-08 — Pedido conserva su historial · docs/capturas/aplicacion/PED-08-pedido-conserva-historial.png



Problems Output Debug Console Terminal Ports node

```
PS C:\Users\Admin\OneDrive - TUPAD UTN\Escritorio\foodstore-tpi-bd2> npm start
Selecciona una opción: 2

===== PRODUCTOS =====
1. Crear producto
2. Listar productos activos
3. Buscar producto activo por ID
4. Modificar producto
5. Desactivar producto
0. Volver al menú principal
Selecciona una opción: 4

Conexión con MongoDB establecida.
ID del producto: 6ad810c81620b9fc03147dfe
```

(index)	id	nombre	categoria	precioARS	activo
0	'6ad810c81620b9fc03147dfe'	'Hamburguesa Prueba'	'comidas'	'1250.50'	true

```
Presiona Enter para conservar el valor actual.
Nuevo nombre:  Hamburguesa Prueba Actualizada
Nueva categoría (comidas, bebidas o postres):
Nuevo precio en ARS: 1400.00

Producto modificado correctamente.

===== PRODUCTOS =====
1. Crear producto
2. Listar productos activos
3. Buscar producto activo por ID
4. Modificar producto
5. Desactivar producto
0. Volver al menú principal
Selecciona una opción: 3
ID del producto: 6ad810c81620b9fc03147dfe
```

(index)	id	nombre	categoria	precioARS	activo
0	'6ad810c81620b9fc03147dfe'	'Hamburguesa Prueba Actualizada'	'comidas'	'1400.00'	true

FIGURA 28 · PED-08 — Producto actualizado · docs/capturas/aplicacion/PED-08-producto-actualizado.png

```
5. Desactivar producto
0. Volver al menú principal
Selecciona una opción: 0

===== MENÚ PRINCIPAL =====
1. Gestionar clientes
2. Gestionar productos
3. Gestionar pedidos
0. Salir
Selecciona una opción: 3

===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción: 3
ID del pedido: 6ad8177af44effa3cac4d7f5
```

(index)	id	clienteId	fecha	totalARS	estado	activo
0	'6ad8177af44effa3cac4d7f5'	'6ad80f5f1620b9fc03147dfd'	'2026-10-21T01:38:04.582Z'	'750.25'	'cancelado'	true

Detalle del pedido:

(index)	productoId	producto	precioUnitarioARS	cantidad	subtotalARS
0	'6ad810e11620b9fc03147dff'	'Bebida Prueba'	'750.25'	1	'750.25'

```
===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción:
```

FIGURA 29 · PED-09 — Historial tras baja de producto · docs/capturas/aplicacion/PED-09-historial-tras-baja-producto.png

```
===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción: 4
ID del pedido: 6ad814da1620b9fc03147e01
```

(index)	id	clienteId	fecha	totalARS	estado	activo
0	'6ad814da1620b9fc03147e01'	'6ad80f5f1620b9fc03147dfd'	'2026-10-21T01:27:18.848Z'	'3251.25'	'pendiente'	true

```
Nuevo estado (entregado/cancelado): entregado

Estado actualizado a entregado.

===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción: 3
ID del pedido: 6ad814da1620b9fc03147e01
```

(index)	id	clienteId	fecha	totalARS	estado	activo
0	'6ad814da1620b9fc03147e01'	'6ad80f5f1620b9fc03147dfd'	'2026-10-21T01:27:18.848Z'	'3251.25'	'entregado'	true

Detalle del pedido:

(index)	productoId	producto	precioUnitarioARS	cantidad	subtotalARS
0	'6ad810c81620b9fc03147dfe'	'Hamburguesa Prueba'	'1250.50'	2	'2501.00'
1	'6ad810e11620b9fc03147dff'	'Bebida Prueba'	'750.25'	1	'750.25'

```
===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción:
```

FIGURA 30 · PED-10 — Pedido entregado · docs/capturas/aplicacion/PED-10-pedido-entregado.png

```
===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción: 4
ID del pedido: 6ad814da1620b9fc03147e01
```

(index)	id	clienteId	fecha	totalARS	estado	activo
0	'6ad814da1620b9fc03147e01'	'6ad80f5f1620b9fc03147dfd'	'2026-10-21T01:27:18.848Z'	'3251.25'	'entregado'	true

```
Solo se puede cambiar el estado de un pedido pendiente.

===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción:
```

FIGURA 31 · PED-12 — Transición bloqueada · docs/capturas/aplicacion/PED-12-transicion-bloqueada.png

```
Bienvenido a Food Store
Trabajo Práctico Integrador - Bases de Datos II

===== MENÚ PRINCIPAL =====
1. Gestionar clientes
2. Gestionar productos
3. Gestionar pedidos
0. Salir
Selecciona una opción: 3

===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción: 5

Conexión con MongoDB establecida.
ID del pedido: 6ad8177af44effa3cac4d7f5
```

(index)	id	clienteId	fecha	totalARS	estado	activo
0	'6ad8177af44effa3cac4d7f5'	'6ad80f5f1620b9fc03147dfd'	'2026-10-21T01:38:04.582Z'	'750.25'	'cancelado'	true

```
¿Confirmas la baja lógica? (s/n): s

Pedido desactivado. El documento y su detalle permanecen en la base.

===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción:
```

FIGURA 32 · PED-13 — Baja lógica de pedido · docs/capturas/aplicacion/PED-13-baja-logica-pedido.png

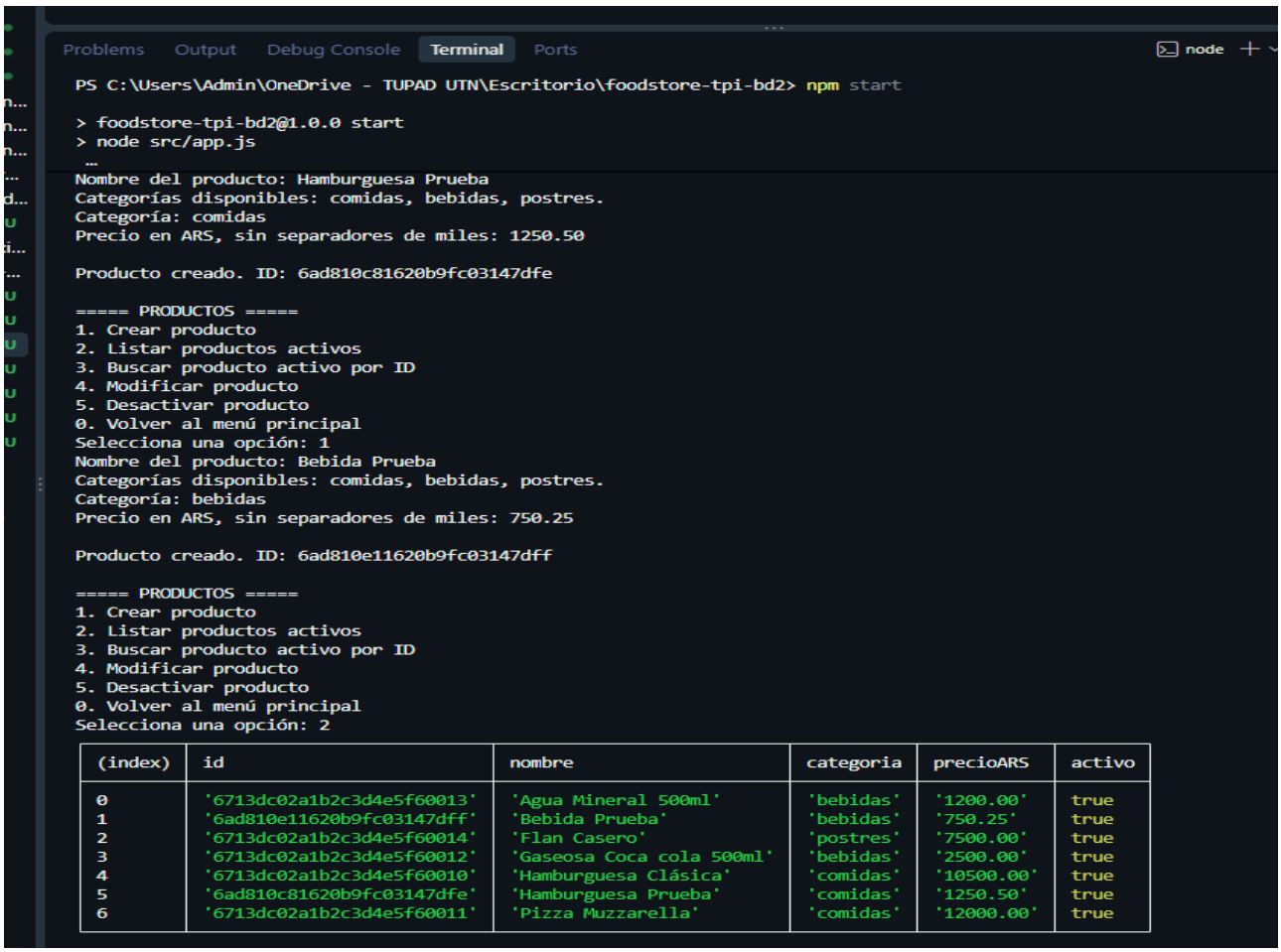


FIGURA 33 · PRO-01 — Productos creados · docs/capturas/aplicacion/PRO-01-productos-creados.png

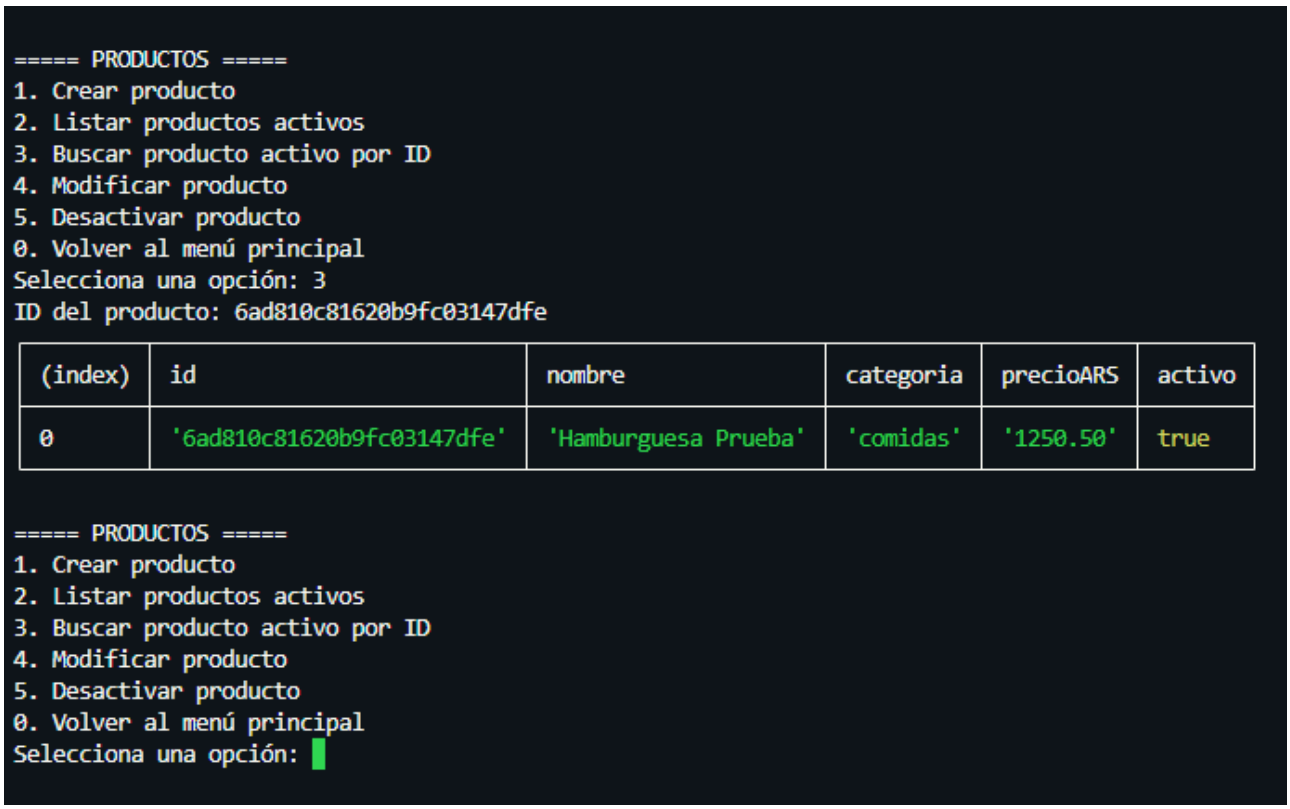


FIGURA 34 · PRO-02 — Búsqueda de producto · docs/capturas/aplicacion/PRO-02-busqueda-producto.png

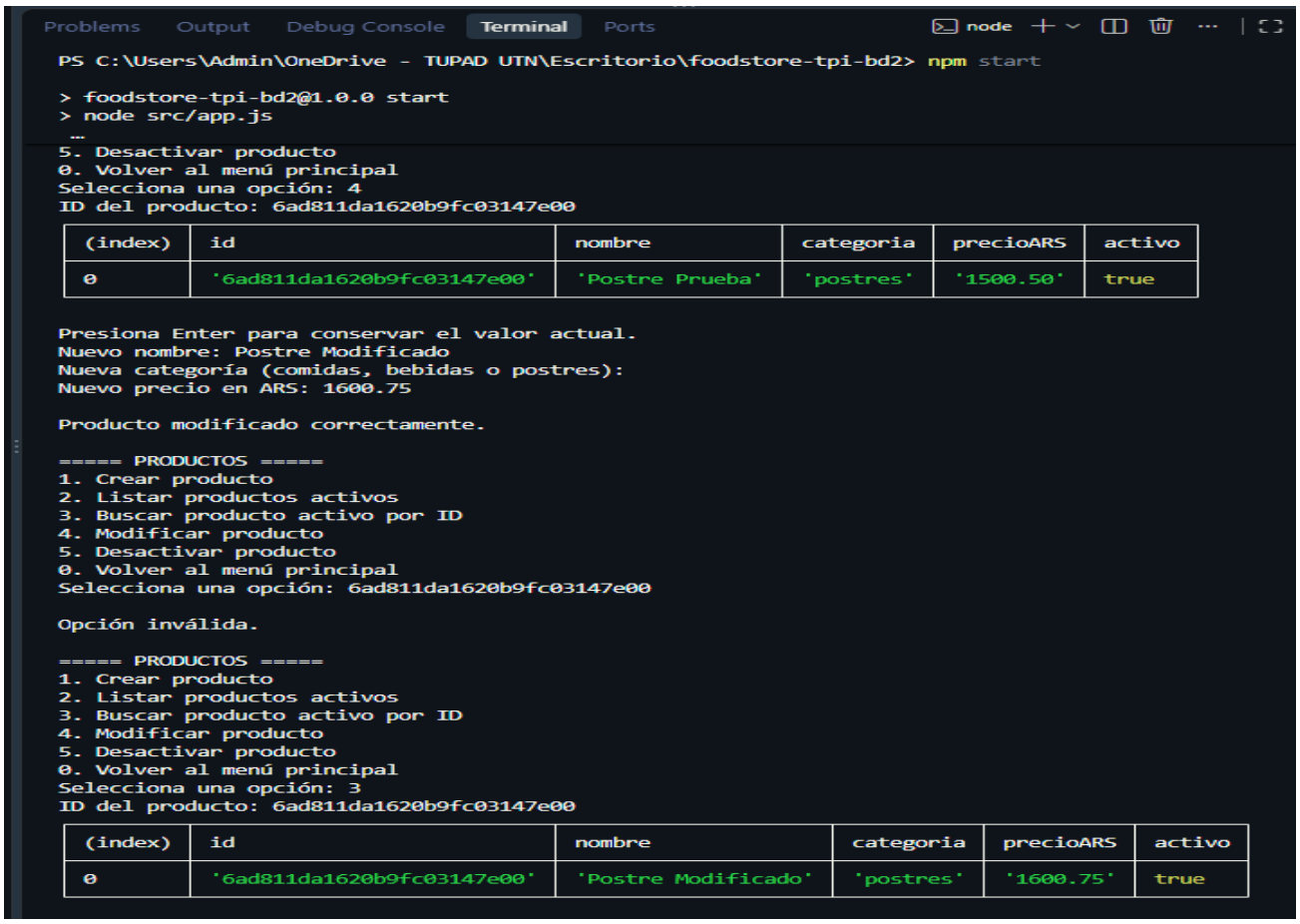


FIGURA 35 · PRO-03 — Producto modificado · docs/capturas/aplicacion/PRO-03-producto-modificado.png

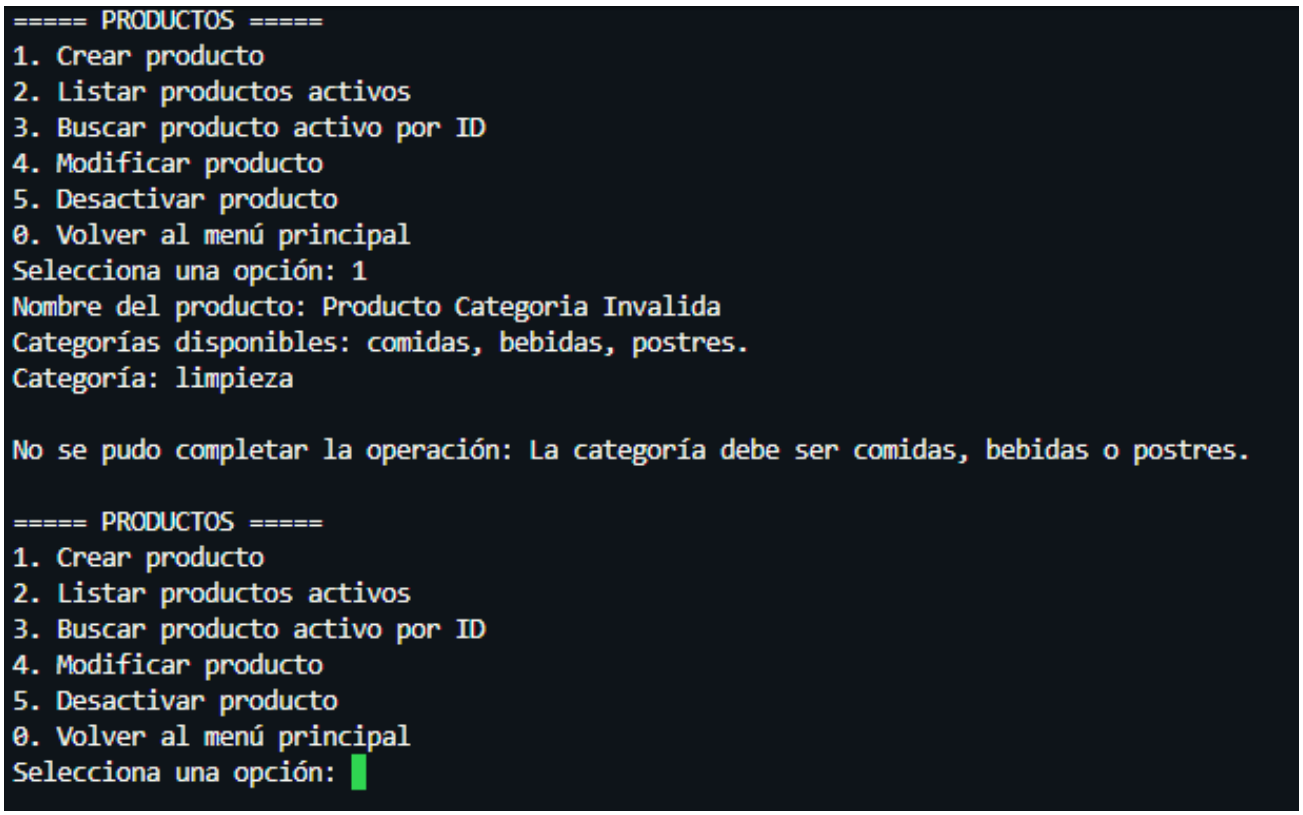


FIGURA 36 · PRO-04 — Categoría inválida · docs/capturas/aplicacion/PRO-04-categoria-invalida.png



FIGURA 37 · PRO-05 — Validación de precios inválidos · docs/capturas/aplicacion/PRO-05-precio-cero-negativo-tresdecimales.png

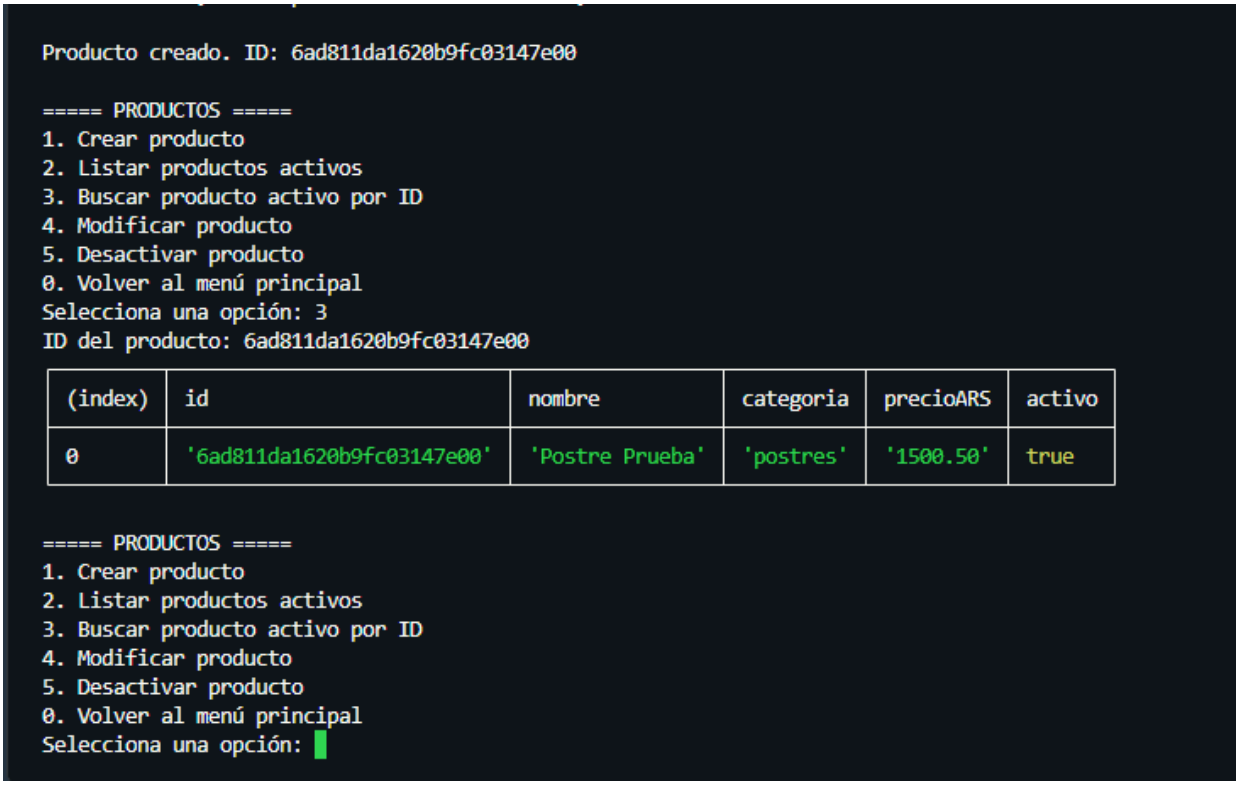


FIGURA 38 · PRO-06 — Precio con coma decimal · docs/capturas/aplicacion/PRO-06-precio-coma.png

Respaldo, restauración y consultas de recuperación

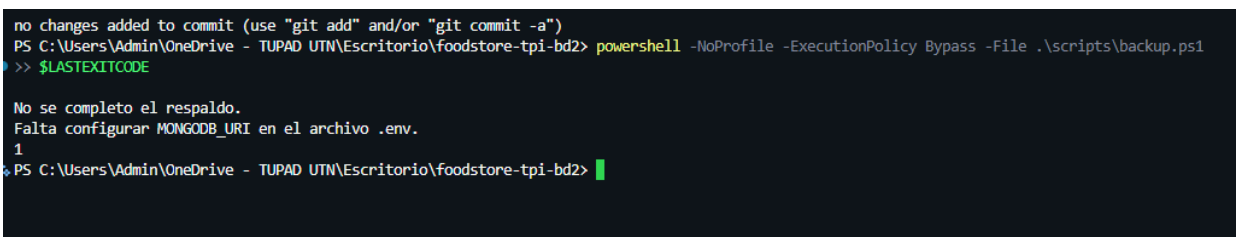


FIGURA 39 · BAK-01 — Error de respaldo controlado · docs/capturas/backup/BAK-01-error-backup.png

```
2026-10-20T22:48:33.060-0300    done dumping `foodstore_tpi.clientes` (5 documents)

Respaldo generado correctamente.
Archivos BSON encontrados: 3
Duracion: 8.21 segundos
Registro: C:\Users\Admin\OneDrive - TUPAD UTN\Escritorio\foodstore-tpi-bd2\resguardos_tpi\2026-10-20\resultado_
backup.txt
0
PS C:\Users\Admin\OneDrive - TUPAD UTN\Escritorio\foodstore-tpi-bd2> Get-ChildItem ".\resguardos_tpi\2026-10-20
\foodstore_tpi"
>> Get-Content ".\resguardos_tpi\2026-10-20\resultado_backup.txt"

    Directory: C:\Users\Admin\OneDrive - TUPAD
UTN\Escritorio\foodstore-tpi-bd2\resguardos_tpi\2026-10-20\foodstore_tpi

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
la---          20/10/2026    22:48            599 clientes.bson
la---          20/10/2026    22:48            266 clientes.metadata.json
la---          20/10/2026    22:48           1762 pedidos.bson
la---          20/10/2026    22:48            174 pedidos.metadata.json
la---          20/10/2026    22:48             51 prelude.json
la---          20/10/2026    22:48            876 productos.bson
la---          20/10/2026    22:48            176 productos.metadata.json
Food Store - Respaldo
Base: foodstore_tpi
Inicio: 2026-10-20 22:48:24
Fin: 2026-10-20 22:48:33
Duracion en segundos: 8.21
Archivos BSON: 3
Codigo de salida de mongodump: 0

PS C:\Users\Admin\OneDrive - TUPAD UTN\Escritorio\foodstore-tpi-bd2>
```

FIGURA 40 · BAK-03 — Archivos del respaldo · docs/capturas/backup/BAK-03-archivos-respaldo.png

```
Base seleccionada para pruebas:
foodstore_tpi_restore_test

> foodstore-tpi-bd2@1.0.0 start
> node src/app.js


Bienvenido a Food Store
Trabajo Práctico Integrador - Bases de Datos II

===== MENÚ PRINCIPAL =====
1. Gestionar clientes
2. Gestionar productos
3. Gestionar pedidos
0. Salir
Selecciona una opción: 1

===== CLIENTES =====
1. Crear cliente
2. Listar clientes activos
3. Buscar cliente activo por ID
4. Modificar cliente
5. Desactivar cliente
0. Volver al menú principal
Selecciona una opción: 2

Conexión con MongoDB establecida.
```

(index)	id	nombre	email	telefono	activo
0	'6713dc01a1b2c3d4e5f60001'	'Celeste Gómez'	'cele.gomez@mail.com'	'1144556677'	true
1	'6ad80f5f1620b9fc03147dfd'	'Cliente Modificado'	'prueba.modificacion@example.com'	'1100000001'	true
2	'6713dc01a1b2c3d4e5f60002'	'Jorge Manzanel'	'jorgitom@mail.com'	'1188990011'	true
3	'6713dc01a1b2c3d4e5f60003'	'Marita Sandoval'	'msandoval@mail.com'	'1122334455'	true

FIGURA 41 · BKP-04 — Validación de clientes recuperados · docs/capturas/backup/BKP-04-validacion-clientes.png

```
Selecciona una opción: 0

===== MENÚ PRINCIPAL =====
1. Gestionar clientes
2. Gestionar productos
3. Gestionar pedidos
0. Salir
Selecciona una opción: 2

===== PRODUCTOS =====
1. Crear producto
2. Listar productos activos
3. Buscar producto activo por ID
4. Modificar producto
5. Desactivar producto
0. Volver al menú principal
Selecciona una opción: 2
```

(index)	id	nombre	categoria	precioARS	activo
0	'6713dc02a1b2c3d4e5f60013'	'Agua Mineral 500ml'	'bebidas'	'1200.00'	true
1	'6713dc02a1b2c3d4e5f60014'	'Flan Casero'	'postres'	'7500.00'	true
2	'6713dc02a1b2c3d4e5f60012'	'Gaseosa Coca cola 500ml'	'bebidas'	'2500.00'	true
3	'6713dc02a1b2c3d4e5f60010'	'Hamburguesa Clásica'	'comidas'	'10500.00'	true
4	'6ad810c81620b9fc03147dfe'	'Hamburguesa Prueba Actualizada'	'comidas'	'1400.00'	true
5	'6713dc02a1b2c3d4e5f60011'	'Pizza Muzzarella'	'comidas'	'12000.00'	true

```
===== PRODUCTOS =====
1. Crear producto
2. Listar productos activos
3. Buscar producto activo por ID
4. Modificar producto
5. Desactivar producto
0. Volver al menú principal
Selecciona una opción:
```

FIGURA 42 · BKP-05 — Validación de productos recuperados · docs/capturas/backup/BKP-05-validacion-productos.png

```
>>
>> try { …
0. Volver al menú principal
Selecciona una opción: 0

===== MENÚ PRINCIPAL =====
1. Gestionar clientes
2. Gestionar productos
3. Gestionar pedidos
0. Salir
Selecciona una opción: 3

===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción: 2
```

(index)	id	clienteId	fecha	totalARS	estado	activo
0	'6ad8177af44effa3cac4d7f5'	'6ad80f5f1620b9fc03147dfd'	'2026-10-21T01:38:04.582Z'	'750.25'	'cancelado'	true
1	'6ad814da1620b9fc03147e01'	'6ad80f5f1620b9fc03147dfd'	'2026-10-21T01:27:18.848Z'	'3251.25'	'entregado'	true
2	'6713dc03a1b2c3d4e5f60022'	'6713dc01a1b2c3d4e5f60003'	'2026-10-18T21:00:00.000Z'	'3600.00'	'pendiente'	true
3	'6713dc03a1b2c3d4e5f60021'	'6713dc01a1b2c3d4e5f60002'	'2026-10-18T20:15:00.000Z'	'27000.00'	'entregado'	true
4	'6713dc03a1b2c3d4e5f60020'	'6713dc01a1b2c3d4e5f60001'	'2026-10-18T19:30:00.000Z'	'23500.00'	'pendiente'	true

```
===== PEDIDOS =====
1. Crear pedido
2. Listar pedidos activos
3. Buscar pedido activo por ID
4. Cambiar estado del pedido
5. Desactivar pedido
0. Volver al menú principal
Selecciona una opción:
```

FIGURA 43· BKP-06 — Validación de pedidos recuperados · docs/capturas/backup/BKP-06-validacion-pedidos.png

ANEXO II · FRAGMENTOS REPRESENTATIVOS DEL CÓDIGO FUENTE

A continuación se presentan extractos auténticos de la implementación que permiten identificar los componentes principales de Food Store. Los fragmentos se muestran con fines explicativos; las funciones completas, las validaciones y el tratamiento de errores se encuentran en los archivos fuente del repositorio y en el ZIP de entrega.

1. Conexión con MongoDB Atlas

Archivo: `src/config/conexion.js`

El driver MongoDB crea la conexión a partir de las variables de entorno y verifica el acceso a Atlas mediante ping.

```
const uri = process.env.MONGODB_URI?.trim();
const nombreBase = process.env.MONGODB_DB?.trim();

cliente = new MongoClient(uri, {
  serverSelectionTimeoutMS: 10000
});
await cliente.connect();
const db = cliente.db(nombreBase);
await db.command({ ping: 1 });
baseDeDatos = db;
```

2. Operaciones CRUD y baja lógica

Archivo: `src/operaciones/clientes.js`

Los siguientes extractos provienen de funciones distintas del módulo: alta, listado, modificación y baja lógica. Se presentan para explicar las operaciones, no para ejecutarse como un único bloque.

```
// CREATE
const resultado = await coleccion.insertOne({
  nombre, email, telefono, activo: true
});

// READ
const clientes = await coleccion
  .find({ activo: true })
  .sort({ nombre: 1 })
  .toArray();

// UPDATE
const resultado = await coleccion.updateOne(
  { _id: id, activo: true },
  { $set: cambios }
);

// DELETE lógico
const resultado = await coleccion.updateOne(
  { _id: id, activo: true },
  { $set: { activo: false } }
);
```

3. Embedding y cálculo de pedidos

Archivo: `src/operaciones/pedidos.js`

El arreglo items guarda las referencias y los datos del producto utilizados en la compra; los importes se calculan con precisión decimal.

```
const subtotal = precio.times(item.cantidad);

items.push({
  productoId: producto._id,
  nombreProducto: producto.nombre,
  precioUnitario: convertirImporte(precio),
  cantidad: new Int32(item.cantidad),
  subtotal: convertirImporte(subtotal)
});

total = total.plus(subtotal);
```

4. Respaldo automatizado por script

Archivo: `scripts/backup.ps1`

El script genera una carpeta distinta para cada fecha y hora y ejecuta mongodump sobre toda la base de datos.

```
$raizProyecto = Split-Path -Parent $PSScriptRoot
$fechaActual = $momentoActual.ToString("yyyy-MM-dd")
$horaActual = $momentoActual.ToString("HH-mm-ss")

$carpetaResguardos = Join-Path $raizProyecto "resguardos_tpi"
$carpetaDia = Join-Path $carpetaResguardos $fechaActual
$carpetaFecha = Join-Path $carpetaDia $horaActual

& $herramienta.Source `
    "--uri=$uriMongo" `
    "--db=$nombreBase" `
    "--out=$carpetaFecha"
$codigoSalida = $LASTEXITCODE
```

5. Índice único de correo electrónico

Archivo: src/operaciones/clientes.js

El índice impide duplicaciones incluso si el cliente anterior fue desactivado.

```
await candidata.createIndex(
  { email: 1 },
  { unique: true }
);
```

6. Validación de precios y Decimal128

Archivo: src/operaciones/productos.js

Se valida la entrada decimal y se guarda el importe monetario con el tipo BSON apropiado.

```
const normalizado = texto.trim().replace(',', '.');
if (!/^\d+(\.\d{1,2})?$/.test(normalizado)) {
  throw new Error(
    'Ingresa un precio positivo con hasta dos decimales, ' +
    'sin separadores de miles. Ejemplo: 1500.50.'
  );
}
const precio = new Decimal(normalizado);
if (!precio.isFinite() || !precio.greaterThan(0)) {
  throw new Error('El precio debe ser mayor que cero.');
}
// Conversion a BSON (extracto de la misma función)
return Decimal128.fromString(precio.toFixed(2));
```

7. Verificación de referencias activas (linking)

Archivo: src/operaciones/pedidos.js

Antes de registrar el pedido se verifica que el cliente y los productos existan y estén activos.

```
const cliente = await clientes.findOne({
  _id: clienteId,
  activo: true
});

const productosActuales = await productos.find({
  _id: { $in: seleccion.map(item => item.productoId) },
  activo: true
}).toArray();
```

8. Actualización controlada de estados

Archivo: src/operaciones/pedidos.js

El filtro evita modificar pedidos inactivos o que ya dejaron de estar pendientes.

```
const resultado = await coleccion.updateOne(
  { _id: id, activo: true, estado: 'pendiente' },
  { $set: { estado: nuevoEstado } }
);
if (resultado.matchedCount === 0) {
  console.log(
    '\nEl pedido cambió de estado o fue desactivado. Vuelve a consultarlo.'
  );
  return;
}
```